

3DGEER: 3D GAUSSIAN RENDERING MADE EXACT AND EFFICIENT FOR GENERIC CAMERAS

Anonymous authors
Anonymous submission

ABSTRACT

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) achieves an appealing balance between rendering quality and efficiency, but its reliance on approximating 3D Gaussians as 2D projections fundamentally limits accuracy, and this limitation becomes severe under generic large field-of-view (FoV) cameras. Despite a wave of recent extensions, no prior work has attained both *projective exactness* and *real-time efficiency* for general cameras. We close this gap with 3DGEER, a geometrically exact and efficient Gaussian rendering framework. From first principles, we derive a closed-form expression for integrating Gaussian density along a ray, which delivers precise forward rendering and exact differentiable optimization under arbitrary camera models. To make this exactness efficient, we introduce the *Particle Bounding Frustum (PBF)*, which yields tight ray-Gaussian association without any BVH traversal, and the *Bipolar Equiangular Projection (BEAP)*, which unifies FoV representations, accelerates association, and improves reconstruction quality. Experiments on both pinhole and fisheye datasets demonstrate that 3DGEER outperforms prior methods across all metrics, runs $5\times$ faster than existing projective-exact ray-based baselines, and generalizes to wider FoVs unseen during training—establishing a new state of the art in real-time radiance field rendering.

1 INTRODUCTION

Initiated by 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023), volumetric particle rendering methods formulated under splatting (Yu et al., 2024a; Yan et al., 2024; Lu et al., 2024; Kheradmand et al., 2024; Charatan et al., 2024; Mallick et al., 2024) have become a dominant paradigm, prized for their visual fidelity and high rendering speed. This efficiency comes at a cost: it stems from EWA splatting (Zwicker et al., 2001), which uses a local affine approximation to project a 3D Gaussian as a 2D Gaussian on the image plane. This approximation holds only in narrow field-of-view (FoV) scenarios; in wide-FoV settings it breaks down, as nonlinear distortions force the true projection to depart sharply from a symmetric 2D Gaussian—most acutely in the fisheye images that dominate robotics and autonomous driving (Rashed et al., 2021; Courbon et al., 2007; Kumar et al., 2023; Zhang et al., 2016; Yogamani et al., 2019; 2024; Sekkat et al., 2022). Recent works enable differentiable rendering under specific fisheye camera models (Liao et al., 2024) or more general camera models (Huang et al., 2024b), but they remain fundamentally bound by their reliance on 2D projective-space approximations and, crucially, none delivers an exact closed-form solution.

Meanwhile, Celarek et al. (2025) question the necessity of exact rendering, arguing that scaling up the number of Gaussians (up to 1M) suffices to absorb the projective approximation in splatting. This argument does not hold beyond small object-centric scenes. In large-scale reconstructions the required number of Gaussians varies drastically across objects and frustums, making such scaling prohibitively costly. More decisively, we show that projective errors under generic distorted cameras are far larger, and that simply inflating the Gaussian count cannot close the resulting quality gap (see Fig. 5).

A parallel line of work tackles volumetric particle rendering directly in a ray-tracing/marching formulation (Yu et al., 2024b; Condor et al., 2025; Mai et al., 2025; Moënné-Loccoz et al., 2024; Tale-gaonkar et al., 2025; Gu et al., 2025). These approaches natively support large-FoV camera models and are free from projective approximation error. Their Achilles’ heel is efficiency—specifically, the

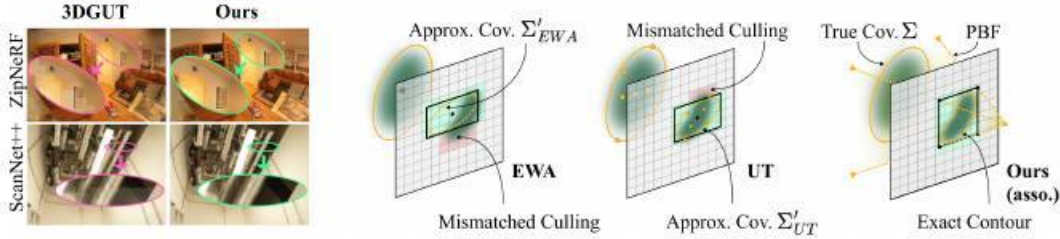


Figure 1: **Linear Approximation Error in Ray-Particle Association.** (Left) Grid-line artifacts produced by inaccurate UT association. (Right) Diagram of our association alongside prior schemes. Our method dispenses with the intermediate conic approximation entirely, computing the exact bounding structure directly from the true 3D covariance.

cost of associating each rendering ray with its most relevant 3D particles. Considerable effort has gone into accelerating this step, notably the advanced Bounding Volume Hierarchies (BVH) used in EVER (Mai et al. 2025) and 3DGRT (Moënné-Loccoz et al. 2024). Yet the inherent algorithmic complexity and poor parallelizability of these structures leave pure ray-based methods well short of 3DGS-level efficiency. The hybrid 3DGUT (Wu et al. 2025) pairs ray-tracing rendering with rasterization to solve ray-particle association efficiently, but it merely relocates the problem: the projective approximation error inherited from the association stage still breaks projective-geometry exactness and can even produce grid-line artifacts (see Fig. 1 Left). In short, no existing method delivers both the geometric exactness of ray-based rendering and the efficiency of splatting.

We resolve this tension with 3DGEER—a 3D Gaussian Exact and Efficient Rendering method that, for the first time, attains *exactness in projective geometry* (see definition in Sec. A) while sustaining *efficiency comparable to splatting* for generic cameras (both perspective and a variety of fish-eye projection models). 3DGEER rests on three contributions: (i) We revisit the heuristic maximum response function (Keselman & Hebert 2022) of Gaussians from first principles and prove its equivalence to projective exactness—a connection left implicit in prior work. Building on this, we derive a differentiable rendering framework that is numerically stable by construction. (ii) To make ray-particle association efficient without sacrificing geometric exactness, we recast the problem as associating Camera Sub-frustums (CSF) with Particle Bounding Frustums (PBF)—the direct 3D analogue of the tile-AABB mapping in 3DGS. For each 3D Gaussian, we derive a closed-form solution for the tight PBF that bounds it, which drives highly efficient association. (iii) We further introduce the novel Bipolar Equiangular Projection (BEAP) space and uniformly sample rays within it to apply color supervision. This strategy not only aligns image-space partitioning with the underlying CSF but also outperforms conventional pinhole and equidistant projections in reconstruction quality.

Extensive experiments on both fisheye and pinhole datasets—including ScanNet++ (Yeshwanth et al. 2023), Zip-NeRF (Barron et al. 2023), Aria (Lv et al. 2025), and MipNeRF (Barron et al. 2021)—establish that 3DGEER consistently outperforms prior and concurrent methods, with 0.9–4.8 PSNR gains under challenging wide-FoV conditions and 0.3–2.0 on pinhole views. Strikingly, our approach surpasses splat-based methods on wide-FoV cameras even when trained only with narrow-FoV data, evidencing exceptional generalization to extremely wide-FoV rendering.

2 RELATED WORKS

2.1 SPLATTING-BASED GAUSSIAN RENDERING

Most 3DGS variants (Yu et al. 2024a, Zhang et al. 2024, Rota Bulò et al. 2024, Gao et al. 2025, Zhao et al. 2025, Kerbl et al. 2024, Hou et al. 2025, Liu et al. 2025) inherit the original EWA splatting strategy (Zwicker et al. 2001; Ren et al. 2002), projecting 3D Gaussians onto the image plane and approximating their appearance as 2D conics. To extend differentiable rendering to distorted large-FoV camera models, methods such as FisheyeGS (Liao et al. 2024) and GS++ (Huang et al. 2024b) estimate the Jacobian of equidistant or spherical projections to correct the 2D mean and covariance of the projected Gaussians. Yet all of these methods still hinge on a first-order Taylor expansion of a highly non-linear projection function. In wide-FoV settings the discarded higher-

Table 1: **Summary of Particle Rendering Methods w.r.t. Projective Exactness.** The top section lists splatting-based methods; the bottom lists ray-based approaches—many of which still fall back on projective-approximated association (e.g., EWA or Unscented Transform). 3DGEER is the only method that simultaneously checks every column.

Method	Render	Particle Asso. Method	Scope	Particle Type	Large FoV	High FPS	Exact Meth.
3DGS (Kerbl et al. 2023)	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
FisheyeGS (Liao et al. 2024)	Splat.	EWA	Img.	3D Gauss.	✓	✓	✗
GS++ (Huang et al. 2024b)	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
2DGS (Huang et al. 2024a)	Ray	Splat Bound.	Img.	2D Gauss.	✗	✓	✓
GOF (Yu et al. 2024b)	Ray	EWA	Img.	3D Gauss.	✗	✗	✗
EVER (Mai et al. 2025)	Ray	BVH	Scene	3D Ellip.	✓	✗	✓
3DGRT (Moënné-Loccoz et al. 2024)	Ray	BVH	Scene	3D Gauss.	✓	✓	✓
3DGUT (Wu et al. 2025)	Ray	UT	Img.	3D Gauss.	✓	✓	✗
HTGS (Hahlbohm et al. 2025)	Ray	Splat Bound.	Img.	3D Gauss.	✗	✓	✓
3DGEER (Ours)	Ray	PBF	Frust.	3D Gauss.	✓	✓	✓

order terms are far from negligible, and the result is substantial approximation error and degraded reconstruction quality.

The root cause is unavoidable: the true projection of a 3D Gaussian under any non-linear camera model—ideal pinhole or distorted alike—is simply not a symmetric 2D Gaussian. Any framework that uses linear projective geometry to approximate an inherently nonlinear transformation therefore incurs approximation error that no amount of tuning can eliminate. Our approach sidesteps this trap entirely, aggregating Gaussian density directly along rays in a volumetric rendering formulation.

2.2 RAY-BASED GAUSSIAN RENDERING

Keselman & Hebert (2022) introduced the assumption that a ray intersects the Gaussian density function at its maximum intensity response. Building on this idea, 3DGRT (Moënné-Loccoz et al. 2024) adopts the heuristic intersection to build a 3D Gaussian ray-tracing framework that samples in ray space without explicit projection and thereby handles wide-FoV distortions. Its cost, however, is prohibitive: the ray-particle association leans on expensive scene-level BVH partitioning structures, and frame rates suffer accordingly.

EVER (Mai et al. 2025) takes a different route, replacing the Gaussian representation with simplified constant-density ellipsoids to obtain a closed-form solution for transmittance-weighted ray integration. This substitution, however, sacrifices the expressive power of full 3D Gaussians and, still relying on BVH-based structures, remains runtime-limited. 2DGS (Huang et al. 2024a) instead replaces 3D Gaussians with 2D surfels to permit exact ray-surface intersections. HTGS (Hahlbohm et al. 2025) reinstates the full Gaussian expressions atop 2DGS’s ray-splat association—but because that association stays confined to screen space, it cannot guarantee projective exactness at wider FoVs. In contrast, we derive closed-form bounds and an entire rendering pipeline directly in a camera-agnostic angular domain (see Sec. D), and we make the computational distinctions from the 2DGS and HTGS parametrizations explicit in Sec. D.3

3DGUT (Wu et al. 2025), a follow-up to 3DGRT, recovers 3DGS-level frame rates through splatting-based particle association. But this acceleration is bought with the Unscented Transform (UT), a sampling-based approximation of 2D conics in projective image space. GOF (Yu et al. 2024b) shares the same weakness, applying the EWA trick directly to fit 2D bounding conics and projective centers for association. Under extreme-FoV conditions these imprecise AABB estimates and the mistaken tile-to-AABB associations they induce surface as visible grid-line artifacts (see Fig. I Left).

Our approach breaks from all of these: we solve ray-particle association directly at the frustum level in 3D space (see Fig. I Right), preserving the geometric exactness of ray-based rendering across arbitrary camera models while matching 3DGS rendering speeds. Tab. I summarizes representative methods targeting projective approximation error and lays out where 3DGEER stands apart along every key dimension.

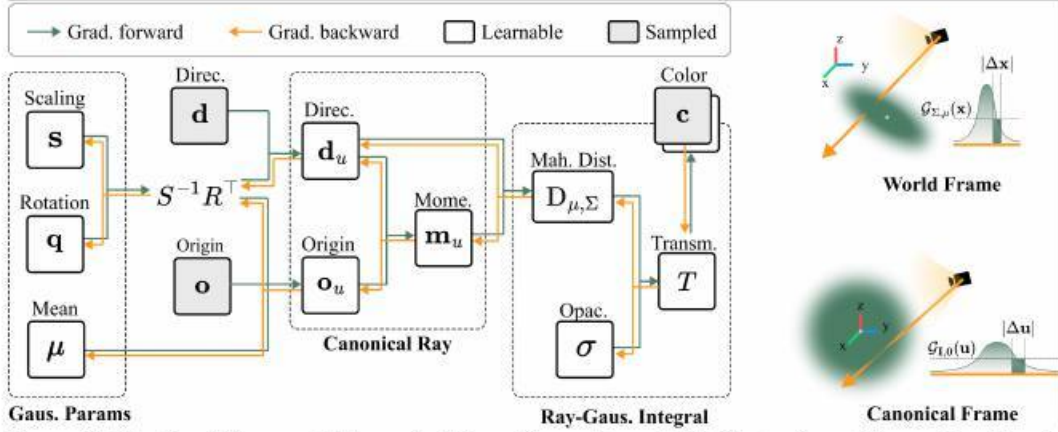


Figure 2: **Gradient Flow and Canonical Transformation.** (Left) Illustration of forward and backward gradient propagation from the Gaussian parameters $\mathbf{s}$, $\mathbf{q}$, $\boldsymbol{\mu}$, and the opacity coefficient σ to the transmittance. (Right) Integral of 3D Gaussian density along a ray, where the green box highlights the product of density and ray segment length (see Eq. B.5).

3 METHOD

We present 3DGEER, a Gaussian rendering framework that delivers both geometric exactness and real-time efficiency under arbitrary FoV cameras. Our method eliminates the two bottlenecks that have held back prior work—*projection approximation* in splatting-based rendering and *BVH overhead* in ray-based rendering—through three complementary components:

- Ray-based rendering formulation (Sec. 3.1): a closed-form differentiable solution for integrating Gaussian density along 3D rays that removes the linearization error of projection-based splatting outright.
- Particle Bounding Frustum (PBF) (Sec. 3.2): a novel ray-particle association scheme that supplants BVH traversal with exact frustum-particle tests, securing accuracy and efficiency at once.
- Bipolar Equiangular Projection (BEAP) (Sec. 3.3): an image representation that unifies arbitrary FoV camera models, accelerating PBF computation while improving reconstruction quality.

The Appendix further provides: (i) the necessary preliminaries and notations (Sec. B.1), (ii) the complete proof to Sec. 3.1 establishing the connection between maximum response and projective exactness (Sec. B), (iii) the full derivations of backward gradients (Sec. C) and its numerical stability, and (iv) the full derivations of geometrically exact association (Sec. D).

3.1 RENDERING WITH EXACT PROJECTIVE GEOMETRY

The central challenge in geometrically exact Gaussian rendering is obtaining a closed-form expression for integrating density and color along a ray. We solve it by mapping each anisotropic Gaussian in world space into a canonical coordinate system in which every Gaussian becomes isotropic. This reduces the transmittance integral to a measure-preserving change of variables, which admits a clean closed-form solution.

Canonical transformation. For each Gaussian with mean $\boldsymbol{\mu}$, rotation R , and scaling S , we define the mapping

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \boldsymbol{\mu} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (1)$$

which maps a canonical coordinate $\mathbf{u}$ to world coordinates $\mathbf{x}$. This implies $\Delta \mathbf{x} = RS \Delta \mathbf{u}$ and absorbs the Jacobian determinant into the density measure (see Fig. 2 Right). Consequently, each Gaussian reduces to the shared isotropic form

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\|\mathbf{u}\|^2\right). \quad (2)$$

Closed-form transmittance. For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, its canonical form is $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ with

$$\mathbf{o}_u = S^{-1}R^\top(\mathbf{o} - \boldsymbol{\mu}), \quad \mathbf{d}_u = S^{-1}R^\top\mathbf{d}. \quad (3)$$

The accumulated transmittance is then

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2}D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d})\right), \quad (4)$$

where

$$D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d}) = \frac{\|\mathbf{o}_u \times \mathbf{d}_u\|^2}{\|\mathbf{d}_u\|^2} \quad (5)$$

is the squared perpendicular (Mahalanobis) distance from the ray to the Gaussian center.

Discussion. This result shows that the transmittance depends solely on the ray’s distance to the Gaussian in canonical space, giving an efficient and geometrically exact closed-form solution. The expression coincides with the “maximum response” heuristic used in prior work (Keselman & Hebert, 2022; Moënné-Loccoz et al., 2024), but our derivation goes further: it proves that this heuristic is in fact projectively exact (see Sec. B). Moreover, our closed-form derivation and its accompanying backward gradient chain (see Fig. 2 Left and Sec. C) are numerically stable by design—they sidestep the instabilities that plague prior ray-based work (Yu et al., 2024b) and remove the need for ad-hoc post-processing such as filtering out degenerate Gaussians.

3.2 RAY-PARTICLE ASSOCIATION WITH EXACT PROJECTIVE GEOMETRY

For any ray-based Gaussian renderer, both accuracy—especially under large-FoV camera models—and runtime efficiency hinge on the ray-particle association strategy, a component prior work has largely neglected. To determine which Gaussians contribute to each ray, we recast the association problem as a mapping from Camera Sub-Frustums (CSFs) to Particle Bounding Frustums (PBFs)—the tight frustums enclosing each Gaussian at a certain distance from its center (Fig. 3). A Gaussian is assigned to a CSF if its PBF intersects the CSF. All Gaussians associated with a CSF are then considered contributors to every ray within that CSF.

To efficiently compute a PBF, we first define two spherical angles (see Fig. E.1) in the camera space:

$$\theta = \arctan\left(\frac{\mathbf{d}_{c,x}}{\mathbf{d}_{c,z}}\right), \quad \phi = \arctan\left(\frac{\mathbf{d}_{c,y}}{\mathbf{d}_{c,z}}\right), \quad (6)$$

where $(\mathbf{d}_{c,x}, \mathbf{d}_{c,y}, \mathbf{d}_{c,z})$ denotes the normalized direction vector pointing to the camera (view) space point $\mathbf{d}_c$. The angles θ and ϕ correspond to the ray’s incidence angles in the horizontal (xz) and vertical (yz) planes, respectively, each ranging from $-\frac{\pi}{2}$ to $\frac{\pi}{2}$. We define a PBF by four planes—two aligned with the θ angle and two with the ϕ angle (see Fig. 3):

$$\mathbf{g}_{\theta_{1,2}} = (-1, 0, \tan \theta_{1,2}, 0)^\top, \quad \mathbf{g}_{\phi_{1,2}} = (0, -1, \tan \phi_{1,2}, 0)^\top, \quad (7)$$

where $\theta_1 < \theta < \theta_2$ and $\phi_1 < \phi < \phi_2$ specify the angular limits.

Given a camera with extrinsic matrix $V = [R_c | \mathbf{t}_c] \in \text{SE}(3)$ and a canonical-to-world transformation matrix H (See Eq. B.4), we apply the inverse transpose of the camera-to-canonical matrix $(VH)^{-1}$ to parameterize the corresponding canonical planes using the angular limits defined in view space. This transformation enables us to effectively derive the constraints for the PBFs. Take the bounds of θ as an example, the canonical plane is given by:

$$\mathbf{g}_{u_{1,2}} = (VH)^\top \mathbf{g}_{\theta_{1,2}} = -(VH)_0 + \tan \theta_{1,2}(VH)_2, \quad (8)$$

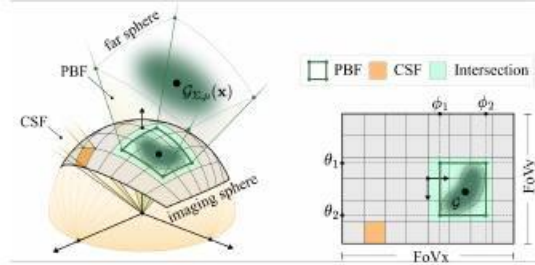


Figure 3: **PBF-CSF Association.** (Left) The PBF, defined by four planes tangent to a 3D Gaussian ellipsoid. (Right) The intersection of the PBF and CSF, unfolded onto the BEAP imaging plane.

where $(VH)_i$ denotes the i -th row of the matrix, expressed as a column vector.

Assuming a Gaussian ellipsoid defined by a λ -standard-deviation contour, we compute the angular bounds of the local frustum by solving the following quadratic equations, which enforce that the transformed planes are tangent to the ellipsoidal contour:

$$((\lambda^2, \lambda^2, \lambda^2, -1) \circ \mathbf{g}_u^\top) \cdot \mathbf{g}_u = 0, \quad (9)$$

where $\circ$ denotes the Hadamard (element-wise) product, and λ is a parameter that controls the spread of the contour, typically set to 3. An identical constraint is applied to $\mathbf{g}_v^\top$ in order to compute the corresponding angular bounds $\phi_{1,2}$. The roots of these quadratic equations determine the tangent values of the angular bounds $[\theta_1, \theta_2]$ and $[\phi_1, \phi_2]$, which define the PBF angular limits. Notably, solving these quadratic equations under canonical space constraints respects the true 3D covariance Σ_c and 3D mean μ_c of the Gaussian in the camera space. Specifically, for the bounds of θ , we have $c = \tan \theta_{1,2}$ satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad \text{where} \quad \mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (10)$$

This formulation is efficient to solve (see Sec. D.1) and dispenses with both the intermediate conic approximations of EWA and UT and the screen-space constraints that cap the FoV of HTGS and 2DGS.

3.3 BIPOLAR EQUIANGULAR PROJECTION (BEAP)

Our BEAP image representation offers three concrete advantages. First, it represents images from large-FoV cameras with no FoV loss. Second, it makes image tiles and their corresponding CSFs share an identical parameterization in terms of two θ and two ϕ angles, which enables highly efficient PBF association and optimal GPU parallelization. Third, the resulting ray sampling is far more uniform in 3D space than conventional projective rendering, and this uniformity directly translates into faster training and higher reconstruction quality.

Specifically, we evenly sample the rays using the spherical angular coordinates (θ, ϕ) (See Eq. 6), and then map the rays back to a discrete $w \times h$ image space using a linear transformation consisting of a scaling factor and a center shift as:

$$(x, y) = \left\lfloor \frac{w\theta}{\text{FoV}_x} + \frac{w+1}{2}, \frac{h\phi}{\text{FoV}_y} + \frac{h+1}{2} \right\rfloor, \quad (11)$$

where $\text{FoV}_x, \text{FoV}_y$ indicate the camera’s horizontal and vertical field of view. As Fig. 4 makes clear, sampling rays uniformly in (θ, ϕ) yields markedly more balanced spatial coverage within the view frustum. This stands in sharp contrast to projective sampling, which squanders samples on peripheral regions, and to equidistant projection, which oversamples the image center.

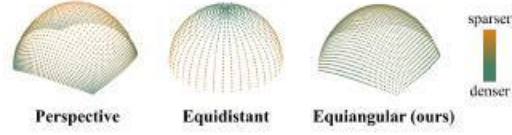


Figure 4: **Comparison of Ray Distributions under Varying Projections.** Projecting pixels from three imaging spaces onto the unit sphere lays the ray distributions side by side. Our BEAP projection attains by far the most uniform coverage.

4 EXPERIMENTS

We evaluate 3DGEER across multiple datasets spanning both pinhole and fisheye camera models. Unless otherwise stated, all experiments run on an RTX 4090 GPU with 64GB of RAM as the default configuration. Full implementation details are provided in Sec. G.

Datasets. We evaluate performance on generic distorted cameras, where projective errors are most damaging, using all scenes from Fisheye ZipNeRF (Barron et al. 2023), five from ScanNet++ (Yeshwanth et al. 2023), and two from Aria (Lv et al. 2025). ZipNeRF/ScanNet++ cameras have 180° diagonal FoV, and Aria has 110° circular FoV. For pinhole settings, we use Pinhole ZipNeRF and MipNeRF360 (Barron et al. 2021).

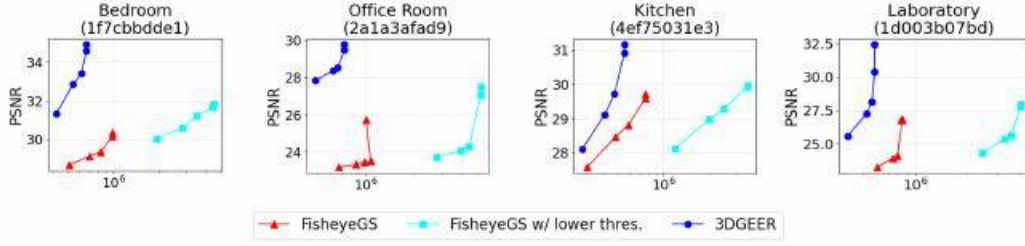


Figure 5: **PSNR $\uparrow$ Trends When Scaling-Up Gaussian Counts.** Brute-force scaling never closes the projective-exactness gap (full metrics in Fig. K.3 and Tab. K.3).



Figure 6: **Extreme-FoV Generalization.** 3DGEER reconstructs complete scenes with fewer Gaussians, whereas FisheyeGS degrades severely.

Evaluation Metrics and Protocol. We report PSNR $\uparrow$, SSIM $\uparrow$, LPIPS $\downarrow$, runtime performance (FPS $\uparrow$), and Gaussian counts $\downarrow$. To eliminate representation bias, all results are projected back to image space for full-FoV evaluation after 30k iterations.

4.1 SCALING GAUSSIANS VS. PROJECTIVE EXACTNESS IN WIDE-FOV CAMERAS

(Celarek et al., 2025) argue that simply scaling the number of Gaussians (up to 1M) suffices to close the linear approximation gap between splat-based and ray-based methods. That claim, however, was only demonstrated on small object-centric scenes and leaves wide-FoV distortions untouched. We test it head-on, scaling the splat-based fisheye method (FisheyeGS) on ScanNet++ scenes by lowering its densification threshold to reach 3.6–5M Gaussians (H200).

The result is unambiguous (Fig. 5): at matched Gaussian counts, the red curve (FisheyeGS) trails the blue curve (3DGEER) by a persistent PSNR margin—a direct signature of the projective-exactness gap (from 4k to 30k iterations, after which densification typically saturates). Even the scaled-up FisheyeGS (light-blue curve), for all its added memory and computation, saturates at 29.3 PSNR, while our method reaches 32.1 with only 550–700K Gaussians. Full statistics appear in Tab. K.3

4.2 COMPREHENSIVE EVALUATION ON ALL-FOV NOVEL VIEW SYNTHESIS

We first evaluate 3DGEER against representative baselines on both fisheye and pinhole benchmarks, adding analysis tailored to each dataset for a comprehensive picture.

ScanNet++. As Tab. 2 shows, 3DGEER outperforms every baseline on every metric. To isolate distortion effects, we evaluate central and peripheral regions separately (details in Sec. G). Competing methods hold up at the center but fall off sharply at the periphery; 3DGEER maintains high fidelity throughout, with sharper boundaries and fewer artifacts (Fig. K.4). Tellingly, even when trained only on central crops, 3DGEER still beats FisheyeGS across all regions.

We further stress-test extreme-FoV generalization (Fig. 6), training on the original dataset FoV (180° diagonal FoV) but testing at much wider FoVs (180° FoV, circular shape). 3DGEER reconstructs

Table 2: **Comparison on the ScanNet++ Dataset.** Each method is trained on either the full FoV or only the central region, then evaluated on the full, central, and peripheral regions to probe robustness under varying distortion. Region-wise metrics are weighted by pixel masks; LPIPS is omitted as it is not directly separable by region (full stats in Tab. K.2).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGS	Central	Persp.	-	-	-	31.26	0.948	-	-
FisheyeGS	Full	Eq.	27.81	0.946	0.139	32.44	0.956	23.28	0.914
EVER	Full	KB	29.47	0.924	0.167	29.93	0.924	28.72	0.925
3DGUT	Full	Eq.	30.64	0.944	0.150	31.87	0.945	28.84	0.937
3DGEER (Ours)	Central	BEAP	29.93	0.949	0.130	32.86	0.954	26.64	0.930
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Table 3: **Comparison on the ZipNeRF Dataset.** A cross-camera generalization setup: each method is trained on either Fisheye (FE, 1/8 resolution) or Pinhole (PH, 1/4 resolution) data and evaluated on both Fisheye and Pinhole test sets (full stats in Tab. K.9).

Train Data Test Data Method	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH
	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.18	26.44	0.858	0.868	0.211	0.239	19.43	26.61	0.791	0.889	0.247	0.212
EVER	25.17	26.14	0.880	0.851	0.153	0.237	22.90	26.45	0.835	0.862	0.207	0.222
3DGUT	24.77	25.59	0.879	0.804	0.183	0.324	18.61	25.96	0.662	0.841	0.300	0.270
3DGEER (Ours)	26.24	27.62	0.897	0.888	0.140	0.214	23.39	27.61	0.846	0.863	0.209	0.254

complete scenes with fewer Gaussians, while splatting-based methods such as FisheyeGS collapse at the periphery (see more extreme-FoV visuals in Fig. K.1 K.2).

ZipNeRF. The dataset provides frame-wise aligned pinhole and fisheye views. Beyond standard fisheye evaluation, we run *cross-camera generalization*—training on fisheye and testing on pinhole, and vice versa—to probe robustness. As Tab. 3 shows, 3DGEER outperforms all methods across metrics, fully exploiting large-FoV training data through geometrically exact rendering. Trained on pinhole, it still delivers notable PSNR gains on both views. FisheyeGS, by contrast, succeeds only in the pinhole-to-pinhole case, and 3DGUT drops sharply on fisheye after pinhole training. Geometric exactness is what separates the field: EVER and 3DGEER generalize well, and 3DGEER is superior in every case. Qualitatively (Fig. 7), 3DGEER recovers finer detail and resists peripheral artifacts most decisively in the hardest pinhole-to-fisheye setting.

We further evaluate 3DGEER on the **MipNeRF360** dataset under standard narrow-FoV conditions—the home turf of splatting methods. Even here, Tab. 4 shows 3DGEER sets the new state of the art across all metrics. It runs over $5\times$ faster than exact ray-marching/tracing methods such as EVER and 3DGRT, and is the only projective-exact method with runtime on par with 3DGS. It also surpasses the projective-approximation baselines (3DGS, 3DGUT) in PSNR, SSIM, and LPIPS alike. Additional quantitative results on **Aria**, an egocentric fisheye dataset (moderate 110° FoV, circular shape) with low-light imaging, along with more visuals, appear in Sec. J.

Table 4: **Quantitative Results on the MipNeRF360 Dataset.** Alongside quality metrics, FPS is reported on an RTX 4090 as the default benchmark. Numbers taken from original works are marked with † (measured on RTX 6000 Ada) and ‡ (measured on RTX 5090).

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
3DGS	27.21	0.815	0.214	343
EVER	27.51	0.825	0.233	36
3DGRT	27.20	0.818	0.248	$52^\dagger / 68^\ddagger$
3DGUT	27.26	0.810	0.218	$265^\dagger / 317^\ddagger$
3DGEER (Ours)	27.76	0.821	0.210	327

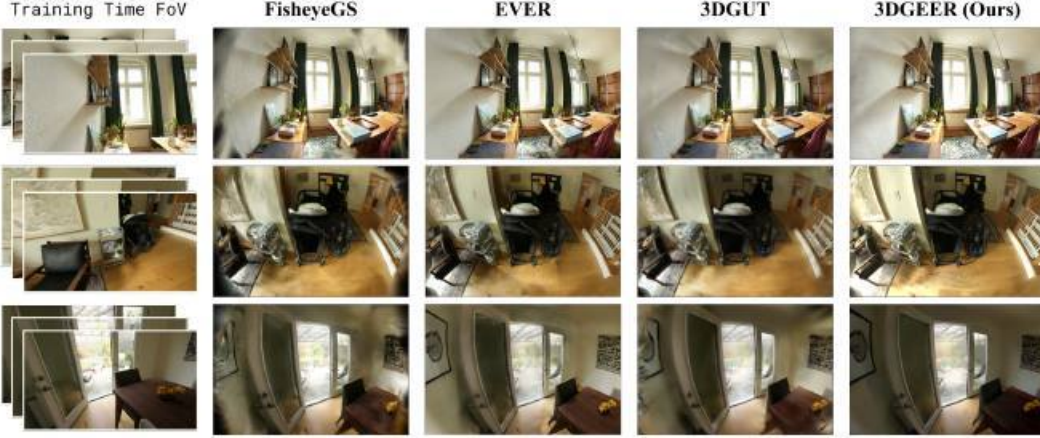


Figure 7: **Qualitative Results on the ZipNeRF Dataset with Cross-Camera Generalization.** The gap is starkest in the most challenging Pinhole (PH) train – Fisheye (FE) test setup, where our method holds a decisive advantage.

Table 5: **Training-Time Transmittance Replacement on ScanNet++.** Splatting-based transmittance inflates the Gaussian count to compensate for its larger approximation error, yet a perceptual gap to our projective-exact formulation persists regardless (full stats in Tab. K.7).

Transmittance Method	Full FoV			Central		Peripheral		Avg. Gaussian Number (k) ↓
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑	
3DGS Splats	22.86	0.878	0.177	27.16	0.884	18.56	0.855	1396.4
FisheyeGS Splats	27.90	0.948	0.141	32.49	0.957	23.36	0.917	920.6
Projective-Exact (Ours)	31.50	0.953	0.126	32.64	0.955	28.94	0.945	591.8

4.3 COMPREHENSIVE RUNTIME ANALYSIS

We analyze runtime on RTX 4090 against splatting-based methods (3DGS on **MipNeRF360** and FisheyeGS on **ScanNet++**). Although ray-based rendering costs slightly more in the rendering stage, PBF’s efficient ray-particle association slashes Gaussian duplication and sorting overhead (detailed runtime in Tab. K.4), so overall efficiency matches optimized splatting pipelines.

The mechanism is clear. PBF produces very tight bounds—each tile associates with only ~ 475 Gaussians on average, $3\text{--}5\times$ fewer than EWA or UT (Tab. K.6)—which directly yields the $2.5\text{--}5\times$ speedup in the association stage (Tab. K.5). Projection-based schemes (EWA, UT) instead overestimate screen-space extent, incurring excessive overdraw and inefficient memory access; even with SnugBox (Hanson et al. 2025) acceleration they stay shackled to miscentered density approximations and often worsen artifacts (Fig. I). Speed is only half the story: PBF also preserves geometric accuracy and numerical stability and eliminates projective approximation error. As Tab. K.6 confirms, even when ray-based rendering Gaussian fields trained under other association schemes, PBF remains artifact-free.

4.4 LOCAL AFFINE APPROXIMATION VS. PROJECTIVE-EXACT INTEGRAL

To isolate the contribution of our projective-exact transmittance formulation (Eq. 4), we swap it out for the commonly used local affine splats (i.e., the 2D conic approximated in 3DGS and FisheyeGS).

Rendering-Time Approx. To sharpen the contrast between the two rendering formulations, we substitute the splatting-based approximation for the transmittance at test time only, keeping every projective-exact component during training. Fig. 8 makes the consequence plain: when transmittance uses the splat approximation but associations follow our exact projective geometry, the mismatch produces incorrect culling. This experiment both magnifies and explains the gridline artifact reported in 3DGUT (Fig. I), where the mismatch runs in the opposite direction—ray-based transmittance paired with conic-approximate association.

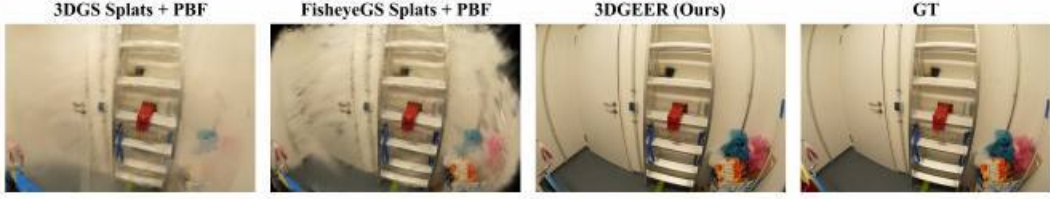


Figure 8: **Rendering-Time Transmittance Func. Replacement.** Swapping projective-exact transmittance for the splatting approximation at test time alone already induces mismatched culling.

Table 6: **BEAP vs. Alternative Supervision Schemes.** BEAP outperforms every other projection on every metric on ScanNet++ (full stats in Tab. K.8).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGEER w/o BEAP	Central	Persp.	29.84	0.943	0.131	32.21	0.951	26.23	0.915
3DGEER w/o BEAP	Full	Persp.	21.11	0.853	0.300	21.32	0.855	20.46	0.846
3DGEER w/o BEAP	Full	Eq.	31.05	0.948	0.135	32.21	0.952	28.56	0.935
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Training-Time Approx. Tab. 5 (on ScanNet++) tells the rest. When splatting-based transmittance is also used during training while association stays projective-exact, the optimizer and densification only partly absorb the approximation error and the cross-view inconsistency splatting introduces. The larger loss drives stronger gradients throughout training and, in turn, 1.6–2.4 $\times$ more Gaussians to fit the scene. Yet even with this inflated Gaussian count, the perceptual gap remains—clear evidence that densification alone cannot correct the approximation error.

4.5 BEAP ON/OFF STUDY

We compare BEAP supervision against other widely used projection models on **ScanNet++**. Here we hold the projective-exact integral and PBF fixed for Gaussian transmittance and association, varying only the ray-sampling distribution used for supervision (pinhole, equidistant, and BEAP). As Tab. 6 and Fig. K.5 show, BEAP supervision wins on every metric against every alternative. Its more uniform sampling supplies smoother, more consistent gradients that converge better on fine details.

The gap widens further under pinhole cameras. At a fixed resolution, a well-balanced information distribution matters more than sheer FoV. In fact, training in the perspective domain, the full FoV can underperform the central region (Tab. 6 rows 1–2): large-FoV images (Fig. K.2 Left) sample angles highly non-uniformly, concentrating informative content at the center while the sparse periphery dominates the image area and is disproportionately oversampled.

5 CONCLUSIONS

We introduced 3DGEER, the first complete, closed-form solution for projective-exact volumetric Gaussian rendering derived from first principles. Around this formulation we built an efficient PBF computation for ray-particle association that delivers high-speed rendering without surrendering projective-geometry exactness, together with the BEAP image representation that enables effective color supervision under wide-FoV camera models. Extensive experiments establish that 3DGEER consistently outperforms the state of the art across multiple datasets, and its advantage is most pronounced precisely where prior methods struggle most—challenging wide-FoV scenarios. 3DGEER thus resolves the long-standing tension between exactness and efficiency in real-time radiance field rendering.

ETHICS STATEMENT

This work builds on publicly available datasets (ScanNet++, ZipNeRF, Aria, and MipNeRF) and does not involve any human subjects, private data, or personally identifiable information. The methods developed are intended for advancing 3D scene reconstruction and rendering research, with potential applications in XR, robotics and autonomous driving. We do not foresee direct harmful consequences, but we note that any misuse of high-fidelity 3D reconstruction techniques could raise privacy concerns. To mitigate this, we restrict our experiments to research datasets that are widely used in the vision community.

REPRODUCIBILITY STATEMENT

We make significant efforts to ensure reproducibility. All mathematical derivations and proofs are provided in the Appendix (Secs. B, C and D). The datasets used in our experiments are publicly available (ScanNet++, ZipNeRF, Aria, and MipNeRF). Details of preprocessing, training protocols, and evaluation metrics are described in Sec. G. Our implementation will be released with training and evaluation scripts upon publication to further facilitate reproducibility.

REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *IEEE International Conference on Computer Vision, (ICCV)*, 2023.
- Adam Celarek, George Kopanas, George Drettakis, Michael Wimmer, and Bernhard Kerbl. Does 3d gaussian splatting need accurate volumetric rendering? In *Computer Graphics Forum (CGF)*, 2025.
- David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 19457–19467, 2024.
- Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrián Jarabo. Don’t splat your gaussians: Volumetric ray-traced primitives for modeling and rendering scattering and emissive media. *ACM Transactions on Graphics (TOG)*, 2025.
- Jonathan Courbon, Youcef Mezouar, Laurent Eckt, and Philippe Martinet. A generic fisheye camera model for robotic applications. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, (IROS)*, pp. 1683–1688, 2007.
- Daniel Duckworth, Peter Hedman, Christian Reiser, Peter Zhizhin, Jean-François Thibert, Mario Lucic, Richard Szeliski, and Jonathan T. Barron. SMERF: streamable memory efficient radiance fields for real-time large-scene exploration. *ACM Transactions on Graphics (TOG)*, 2024.
- Zhongpai Gao, Benjamin Planche, Meng Zheng, Anwesa Choudhuri, Terrence Chen, and Ziyang Wu. 6dgs: Enhanced direction-aware gaussian splatting for volumetric rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10943–10952, 2025.
- Yuliang Guo, Sparsh Garg, S. Mahdi H. Miangoleh, Xinyu Huang, and Liu Ren. Depth any camera: Zero-shot metric depth estimation from any camera. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

-
- Florian Hahlbohm, Fabian Friederichs, Tim Weyrich, Linus Franke, Moritz Kappel, Susana Castillo, Marc Stamminger, Martin Eisemann, and Marcus Magnor. Efficient perspective-correct 3d gaussian splatting using hybrid transparency. In *Computer Graphics Forum (CGF)*, 2025.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 21537–21546, 2025.
- Qiqi Hou, Randall Rauwendaal, Zifeng Li, Hoang Le, Farzad Farhadzadeh, Fatih Porikli, Alexei Bourd, and Amir Said. Sort-free gaussian splatting via weighted sum rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In Andres Burbano, Denis Zorin, and Wojciech Jarosz (eds.), *ACM SIGGRAPH Conference Papers*, 2024a.
- Letian Huang, Jiayang Bai, Jie Guo, and Yanwen Guo. Gs++: Error analyzing and optimal gaussian splatting. *CoRR*, 2024b.
- Juho Kannala and Sami S Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2006.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 2023.
- Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Leonid Keselman and Martial Hebert. Approximate differentiable rendering with algebraic surfaces. In *European Conference on Computer Vision (ECCV)*, 2022.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:80965–80986, 2024.
- Varun Ravi Kumar, Ciarán Eising, Christian Witt, and Senthil Kumar Yogamani. Surround-view fisheye camera perception for automated driving: Overview, survey & challenges. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):3638–3659, 2023.
- Zimu Liao, Siyan Chen, Rong Fu, Yi Wang, Zhongling Su, Hao Luo, Li Ma, Linning Xu, Bo Dai, Hengjie Li, et al. Fisheye-gs: Lightweight and extensible gaussian splatting module for fisheye cameras. *ECCV Workshop*, 2024.
- Changkun Liu, Shuai Chen, Yash Sanjay Bhalgat, Siyan HU, Ming Cheng, Zirui Wang, Victor Adrian Prisacariu, and Tristan Braud. Gs-cpr: Efficient camera pose refinement via 3d gaussian splatting. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20654–20664, 2024.
- Zhaoyang Lv, Maurizio Monge, Ka Chen, Yufeng Zhu, Michael Goesele, Jakob Engel, Zhao Dong, and Richard Newcombe. Photoreal scene reconstruction from an egocentric device. In *ACM SIGGRAPH Conference Papers*, 2025.
- Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T Barron, and Yinda Zhang. EVER: exact volumetric ellipsoid rendering for real-time view synthesis. *IEEE International Conference on Computer Vision (ICCV)*, 2025.

- Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.
- Xiaoyang Mao, Lichan Hong, and Arie Kaufman. Splatting of curvilinear volumes. In *Proceedings Visualization '95*, pp. 61–68. IEEE, 1995.
- Christopher Mei and Patrick Rives. Single view point omnidirectional camera calibration from planar grids. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2007.
- Nicolas Moënné-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 2024.
- Hazem Rashed, Eslam Mohamed, Ganesh Sistu, Varun Ravi Kumar, Ciaran Eising, Ahmad El-Sallab, and Senthil Yogamani. Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, (WACV), pp. 2272–2280, 2021.
- Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer graphics forum (CGF)*, 2002.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 347–362. Springer, 2024.
- Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Ahmed Rida Sekkat, Yohan Dupuis, Varun Ravi Kumar, Hazem Rashed, Senthil Yogamani, Pascal Vasseur, and Paul Honeine. Synwoodscape: Synthetic surround-view fisheye camera dataset for autonomous driving. *IEEE Robotics and Automation Letters (RAL)*, 7(3):8502–8509, 2022.
- Chinmay Talegaonkar, Yash Belhe, Ravi Ramamoorthi, and Nicholas Antipa. Volumetrically consistent 3d gaussian rasterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10953–10963, 2025.
- Lee Westover. Footprint evaluation for volume rendering. In *Proceedings of the 17th annual conference on Computer graphics and interactive techniques*, pp. 367–376, 1990.
- Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moënné-Loccoz, and Zan Gojcic. 3dgt: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3d gaussian splatting for anti-aliased rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 20923–20931, 2024.
- Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Senthil Yogamani, Ciarán Hughes, Jonathan Horgan, Ganesh Sistu, Padraig Varley, Derek O’Dea, Michal Uricár, Stefan Milz, Martin Simon, Karl Amende, et al. Woodscape: A multi-task, multi-camera fisheye dataset for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9308–9318, 2019.
- Senthil Yogamani, David Unger, Venkatraman Narayanan, and Varun Ravi Kumar. Fisheyebevseg: Surround view fisheye cameras based bird’s-eye view segmentation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 1331–1334, 2024.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, (CVPR), pp. 19447–19456, 2024a.

-
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)*, 2024b.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 326–342. Springer, 2024.
- Zichao Zhang, Henri Rebecq, Christian Forster, and Davide Scaramuzza. Benefit of large field-of-view cameras for visual odometry. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 801–808. IEEE, 2016.
- Hexu Zhao, Haoyang Weng, Daohan Lu, Ang Li, Jinyang Li, Aurojit Panda, and Saining Xie. On scaling up 3d gaussian splatting training. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pp. 29–538. IEEE, 2001.

756	CONTENTS	
757		
758	1 Introduction	1
759		
760	2 Related Works	2
761		
762	2.1 Splatting-Based Gaussian Rendering	2
763		
764	2.2 Ray-Based Gaussian Rendering	3
765		
766	3 Method	4
767		
768	3.1 Rendering with Exact Projective Geometry	4
769		
770	3.2 Ray-Particle Association with Exact Projective Geometry	5
771		
772	3.3 Bipolar Equiangular Projection (BEAP)	6
773		
774	4 Experiments	6
775		
776	4.1 Scaling Gaussians vs. Projective Exactness in Wide-FoV Cameras	7
777		
778	4.2 Comprehensive Evaluation on All-FoV Novel View Synthesis	7
779		
780	4.3 Comprehensive Runtime Analysis	9
781		
782	4.4 Local Affine Approximation vs. Projective-Exact Integral	9
783		
784	4.5 BEAP On/Off Study	10
785		
786	5 Conclusions	10
787		
788	a Ethics Statement	11
789		
790	b Reproducibility Statement	11
791		
792	c LLM Usage Statement	17
793		
794	A Appendix: Exactness and Limitation	17
795		
796	B Appendix: Ray-Gaussian Transmittance with Projective Exactness	17
797		
798	B.1 Preliminaries And Notations	17
799		
800	B.2 Differentiable Rendering with Projective Exactness	18
801		
802	B.3 Ray Integral of Isotropic 3D Gaussian	19
803		
804	C Appendix: Cookbook of Gradient Computation	20
805		
806	C.1 Chain Rule	20
807		
808	C.2 Two-Stage Propagation	20
809		
	C.3 Gradients for Canonical Ray	20
	C.4 Gradients for Whitening Matrix	21
	C.5 Gradients for Scaling and Rotation	21
	C.6 Gradients for 3D Mean	21
	C.7 Numerical Stability of Our Differentiable Framework	21

810	D Appendix: Additional Details in Association	22
811	D.1 Particle Bounding Frustum (PBF) Solver	22
812	D.2 Details in Computing CSF-PBF Intersection	23
813	D.3 Relation to Prior Works	24
814		
815	E Appendix: BEAP Image Inverse Interpolation	25
816	Pinhole Camera Model	25
817	KB Fisheye Model	25
818		
819	F Appendix: High-level Algorithmic Summary	26
820		
821	G Appendix: Additional Implementation Details	26
822	Baselines	26
823	Implementation Details	26
824	Central and peripheral regions in ScanNet++ evaluation	27
825	Cross-camera generalization evaluation on ZipNeRF	27
826		
827	H Appendix: EWA Splatting Preliminaries	27
828	From camera coordinates to ray coordinates	27
829	Local affine approximation	27
830	Splatting approximation error	28
831		
832	I Appendix: Advantage over Prior Ray-Tracing Methods	28
833		
834	J Appendix: Results on Egocentric Dataset	29
835		
836	K Appendix: Additional Material and Results	30
837	K.1 Additional Material.	30
838	K.2 Additional Results.	30
839		
840		
841		
842		
843		
844		
845		
846		
847		
848		
849		
850		
851		
852		
853		
854		
855		
856		
857		
858		
859		
860		
861		
862		
863		

LLM USAGE STATEMENT

We used a large language model (ChatGPT, OpenAI) as a writing assistant. Specifically, the LLM was employed to (i) improve the clarity and readability of the manuscript, (ii) suggest stylistic edits to figure captions and section transitions, and (iii) provide alternative phrasings for technical descriptions without altering the scientific content.

The LLM was not involved in research ideation, algorithm design, data analysis, or experimental validation. All technical contributions, mathematical derivations, and results are the work of the authors. The authors take full responsibility for the content of this paper.

A APPENDIX: EXACTNESS AND LIMITATION

We state our claim precisely: our exactness means *eliminating the linear approximation in projective geometry* in both the “rendering” and “ray-particle association” procedures—an error that, unlike splatting, does not shrink as the number of Gaussians grows (see Sec. 4.1). We are deliberate about scope: we do not claim exactness in the full physical volumetric sense, as our framework still involves approximations of ordering, self-attenuation, and overlapping, which we discuss as limitations below.

Limitations. Our formulation integrates each Gaussian independently; hence it does not explicitly resolve self-occlusion, and ordering is handled through depth-based sorting. As also shown in the supplementary videos, this may lead to occasional popping artifacts. To our knowledge, EVER (Mai et al. 2025) is the only method that analytically resolves ordering and self-occlusion by approximating each Gaussian as a constant-density ellipsoid, though this comes with a substantial reduction in rendering speed.

Crucially, this limitation is shared by the entire family of ray-tracing-based Gaussian renderers: Fuzzy Metaballs (Keselman & Hebert 2022) and its successors, including 3DGR/UT (Moënnelocoz et al. 2024; Wu et al. 2025), all rely on the same “maximum-response” assumption for ray-Gaussian interaction. That assumption ignores ordering and self-attenuation by construction, yet has been shown to introduce negligible error in practice (see Celarek et al. 2025 Fig. 9-11 / Sec. 7.1) regardless of the number of Gaussians. Our depth-based ordering therefore attains the same practical accuracy as these ray-tracing-based renderers while retaining our projective-exactness advantage.

Future work. The practical impact of ordering and self-occlusion is small in current Gaussian-based ray tracing, but these factors do bound exact volumetric correctness. One promising direction is to incorporate analytic or learnable self-attenuation models—inspired by ellipsoidal integration (e.g., EVER (Mai et al. 2025)) but without its heavy runtime cost. Another is to handle ordering more principledly, which should further suppress the rare popping artifacts seen under extreme configurations. These limitations are orthogonal to our contributions on projective exactness, and resolving them would only broaden the framework’s reach to large-scale and high-dynamic scenes.

B APPENDIX: RAY-GAUSSIAN TRANSMITTANCE WITH PROJECTIVE EXACTNESS

B.1 PRELIMINARIES AND NOTATIONS

We follow the classic parameterization of 3D Gaussian primitives as introduced in prior work (Zwicker et al. 2001). Each 3D Gaussian distribution is defined by a mean position $\mu \in \mathbb{R}^3$ and a 3D covariance matrix Σ in the world coordinate system:

$$\mathcal{G}_{\Sigma, \mu}(\mathbf{x}) = \frac{1}{\rho |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right), \quad (\text{B.1})$$

where we adopt the normalization constant ρ as $\sqrt{2\pi}$, and $\mathbf{x} \in \mathbb{R}^3$ denotes a point in world space. This formulation differs from recent methods such as 3DGS (Kerbl et al. 2023) and 3DGUT (Wu et al. 2025), which omit the determinant term $|\Sigma|^{1/2}$ to simplify splatting or association during rendering. The form of our covariance matrix obeys: $\Sigma = R S S^\top R^\top$, given a scaling matrix

$S = \text{diag}(\mathbf{s})$ constructed from the scaling vector $\mathbf{s} \in \mathbb{R}^3$, and a rotation matrix $R \in \text{SO}(3)$ derived from a unit quaternion $\mathbf{q} \in \mathbb{R}^4$.

To evaluate the transmittance contributed by a single 3D Gaussian particle along a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, we have:

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\Sigma, \mu}(\mathbf{o} + t\mathbf{d}) dt, \quad (\text{B.2})$$

where σ is the opacity coefficient associated with the Gaussian.

Finally, considering a set of 3D Gaussian particles sorted by depth contributing to the rendering color of the given ray, the rendered color C is computed as:

$$C(\mathbf{o}, \mathbf{d}) = \sum_i c_i(\mathbf{o}, \mu_i) T_i \prod_{j=1}^{i-1} (1 - T_j), \quad (\text{B.3})$$

where $c_i(\mathbf{o}, \mu_i)$ derives the view-dependent color of the i -th Gaussian as seen from the optical center $\mathbf{o}$, parameterized using spherical harmonics (SH) (Kerbl et al. 2023). Through minimizing the reconstruction loss between the rendered color and the observed color along each ray, the parameters $\mathbf{s}, \mathbf{q}, \mu$, the coefficient σ as well as the SH coefficients, are jointly optimized via backpropagation.

B.2 DIFFERENTIABLE RENDERING WITH PROJECTIVE EXACTNESS

The core challenge in achieving projective-exact Gaussian rendering in a ray marching formulation lies in obtaining a closed-form solution for integrating Gaussian density and color along a 3D ray. To address this, we first transform the ray—relative to each Gaussian—into a canonical coordinate system where the transmittance integral (see Eq. B.2) remains consistent and the Gaussians become isotropic. We then derive the exact ray color from first principles.

Specifically, we define a canonical coordinate system shared across all 3D Gaussian primitives. A point $\mathbf{u} \in \mathbb{R}^3$ in this canonical space is mapped to world coordinates for each Gaussian via:

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = H \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (\text{B.4})$$

From this transformation, an infinitesimal segment $\Delta \mathbf{u}$ along a ray in the canonical frame corresponds to a segment $\Delta \mathbf{x} = RS \Delta \mathbf{u}$ in the world frame.

To ensure that the transmittance contributed by each Gaussian along the ray is preserved across coordinate systems, we require that the elementary transmittance (i.e., the product of density and segment length) remains invariant under the transformation (see Fig. 2 Right):

$$\mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{u}) |\Delta \mathbf{u}| = \mathcal{G}_{\Sigma, \mu}(\mathbf{x}) |\Delta \mathbf{x}|. \quad (\text{B.5})$$

This constraint implies that the Jacobian determinant is absorbed into the measure. Note that $|\frac{\Delta \mathbf{x}}{\Delta \mathbf{u}}| = |RS| = |\Sigma|^{1/2}$. We can thus express the corresponding canonical density as:

$$\mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.6})$$

which is isotropic and shared across all primitives in canonical space.

This leads to an important conclusion that the exact closed-form solution for the transmittance integral of each Gaussian along the ray $\mathbf{r}(t)$ can thus be obtained by computing the accumulated density along the transformed ray $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ in the canonical isotropic Gaussian. Note that $t < 0$ is always negligible unless the optical center is within 3-sigma, then all 3D Gaussian rendering frameworks cull the Gaussian when optimization.

This admits a *closed-form* solution in terms of the perpendicular distance from the ray to the Gaussian center, as captured in Theorem 1

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.7})$$

where $D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d})$ denotes the perpendicular distance from the Gaussian center (i.e., the origin in the canonical space) to the transformed ray $\mathbf{r}_u(t)$. Intuitively, this quantity also represents the minimal *Mahalanobis distance* from any points on the ray to the Gaussian in world space, measuring how close the ray passes by the particle distribution.

The remaining question is just about how to compute the perpendicular distance from the canonical origin to the transformed ray. We derive the transformed ray's direction vector $\mathbf{d}_u$ and moment vector $\mathbf{m}_u$ as:

$$\mathbf{d}_u = (S^{-1}R^\top) \mathbf{d}, \quad \mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u, \quad (\text{B.8})$$

where $\mathbf{o}_u = (S^{-1}R^\top) (\mathbf{o} - \boldsymbol{\mu})$ denotes the optical center in canonical space. Based on this, the square of $D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d})$ can be efficiently calculated as:

$$(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2 = \frac{\mathbf{m}_u^\top \mathbf{m}_u}{\mathbf{d}_u^\top \mathbf{d}_u}. \quad (\text{B.9})$$

By substituting this distance square into Eq. [B.7](#) and then the derived transmittance value into Eq. [B.3](#) we compute the accumulated color along the ray.

B.3 RAY INTEGRAL OF ISOTROPIC 3D GAUSSIAN

Theorem 1. *Given a ray and a standard isotropic 3D Gaussian, the ray-Gaussian integral has a closed-form proportional to:*

$$\exp\left(-\frac{1}{2}D^2\right),$$

where D is the perpendicular distance from the Gaussian center to the ray.

Proof. We first apply a translation to the coordinate system such that the Gaussian $\mathcal{G}$ is centered at the origin, and the ray $\mathbf{r}_u$ is translated accordingly. Next, we rotate the entire scene so that the ray direction is aligned with the z -axis. Due to the translation-invariance of the integral and the isotropy of the standard Gaussian, these transformations do not affect the result of the integral.

The translated standard isotropic 3D Gaussian is defined as:

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.10})$$

which matches Eq. [B.6](#). Let the z -axis aligned ray as:

$$\mathbf{r}_u(t) = \begin{bmatrix} \mathbf{o}_u \\ 0 \end{bmatrix} + t \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad (\text{B.11})$$

the ray-Gaussian integral becomes:

$$\begin{aligned} \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{r}_u(t)) dt &= \frac{1}{\rho} \exp\left(-\frac{u_0^2 + v_0^2}{2}\right) \int_{t \in \mathbb{R}} \exp\left(-\frac{1}{2}t^2\right) dt \\ &= \frac{\sqrt{2\pi}}{\rho} \exp\left(-\frac{1}{2}D^2\right), \end{aligned} \quad (\text{B.12})$$

where $D^2 = \frac{u_0^2 + v_0^2}{2}$ denotes the squared perpendicular distance from the ray to the Gaussian center in canonical space. Thus, the integral is proportional to $\exp\left(-\frac{1}{2}D^2\right)$, as stated in theorem. $\square$

Note that the density function shared by our 3D primitives in the canonical space (see Eq. [B.6](#)) follows the same formulation as in the isotropic case. Given the world-coordinate ray $(\mathbf{o}, \mathbf{d})$ as the input of D , its value also depends on the world-space Gaussian parameters $\boldsymbol{\mu}, \Sigma$, which are used to transform the ray into the canonical coordinate system. Therefore, for each Gaussian contributing to the ray, we derive the transmittance as:

$$T = \frac{\sqrt{2\pi}\sigma}{\rho} \exp\left(-\frac{1}{2}(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.13})$$

which recovers Eq. [B.7](#) when picking $\rho = \sqrt{2\pi}$.

C APPENDIX: COOKBOOK OF GRADIENT COMPUTATION

C.1 CHAIN RULE

Recall that $T = \sigma\alpha$ is the ray-Gaussian transmittance (Eq. B.7), where σ is the opacity coefficient, and α is the ray-Gaussian integral. We denote $\kappa = D_{\mu, \Sigma}^2$ as the minimal squared Mahalanobis distance from the ray to the Gaussian distribution, S as the Gaussian scaling, and R as the rotation matrix in the world space. Additionally, given $\Sigma = RSS^\top R^\top$ as the 3D covariance matrix of the Gaussian in the world space, we have W_{pca} as the PCA whitening matrix satisfying:

$$W_{\text{pca}} = \Sigma^{-\frac{1}{2}} = S^{-1}R^\top, \quad \text{where } S^{-1} = \text{diag}\left(\frac{1}{s}\right). \quad (\text{C.1})$$

As illustrated in Fig. 2 we can apply the chain rule to find derivatives *w.r.t.* the Gaussian parameters, e.g, scaling vector s , rotation quaternion q , and mean μ :

$$\frac{\partial T}{\partial s} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial s}, \quad (\text{C.2})$$

$$\frac{\partial T}{\partial q} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial q}, \quad (\text{C.3})$$

$$\frac{\partial T}{\partial \mu} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial \mu}, \quad (\text{C.4})$$

C.2 TWO-STAGE PROPAGATION

To reduce computational overhead in shared propagation, we design our gradient computation following the structured two-stage approach of 3DGS (Kerbl et al. 2023), comprising `RenderCUDA` (RC) and `PreprocessCUDA` (PC). The former performs ray-wise accumulation, efficiently accelerated by our Particle Bounding Frustum (PBF) association (see Section 3.2), while the latter operates with Gaussian-wise complexity, leveraging view frustum culling for additional speedup.

According to Eq. B.7 we can respectively write the gradient *w.r.t.* opacity coefficient and the shared partial derivative for Gaussian parameters in RC as:

$$\left. \frac{\partial T}{\partial \sigma} \right|_{\text{RC}} = \alpha, \quad \left. \frac{\partial T}{\partial \kappa} \right|_{\text{RC}} = -\frac{T}{2}. \quad (\text{C.5})$$

The remaining gradients to the Gaussian parameters can then be back-propagated from the squared distance, e.g., $\frac{\partial \kappa}{\partial s}, \frac{\partial \kappa}{\partial q}, \frac{\partial \kappa}{\partial \mu}$.

C.3 GRADIENTS FOR CANONICAL RAY

We then backpropagate gradients to the canonical ray parameters: the origin $\mathbf{o}_u$, direction $\mathbf{d}_u$, and moment $\mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u$, as defined in Eq. B.8 in our paper. These quantities directly influence the Mahalanobis distance in Eq. B.9 and are critical for capturing canonical ray geometry.

$$\left. \frac{\partial \kappa}{\partial \mathbf{m}_u} \right|_{\text{RC}} = \left(\frac{2}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{m}_u, \quad (\text{C.6})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{o}_u} \right|_{\text{RC}} = \frac{\partial \kappa}{\partial \mathbf{m}_u} \times \mathbf{d}_u, \quad (\text{C.7})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{d}_u} \right|_{\text{RC}} = -\left(\frac{2\kappa}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{d}_u + \mathbf{o}_u \times \frac{\partial \kappa}{\partial \mathbf{m}_u}. \quad (\text{C.8})$$

C.4 GRADIENTS FOR WHITENING MATRIX

Recall that $\mathbf{d}_u = W_{\text{pca}} \mathbf{d}$ and $\mathbf{o}_u = W_{\text{pca}}(\mathbf{o} - \boldsymbol{\mu})$, where $\mathbf{o} = -R_c^\top \mathbf{t}_c$ denotes the world space optical center, and $\mathbf{d}$ denotes the ray direction. Once the gradients *w.r.t.* the ray parameters are obtained, we further propagate them to the PCA whitening matrix W_{pca} via outer products with the world-space vectors:

$$\begin{aligned} \text{RenderCUDA: } \frac{\partial \kappa}{\partial W_{\text{pca}}} \Big|_{\text{RC}} &= \frac{\partial \kappa}{\partial \mathbf{d}_u} \mathbf{d}^\top; \\ \text{PreprocessCUDA: } \frac{\partial \kappa}{\partial W_{\text{pca}}} \Big|_{\text{PC}} &= \frac{\partial \kappa}{\partial \mathbf{o}_u} (\mathbf{o} - \boldsymbol{\mu})^\top; \\ \text{Overall: } \frac{\partial \kappa}{\partial W_{\text{pca}}} &= \frac{\partial \kappa}{\partial W_{\text{pca}}} \Big|_{\text{RC}} + \frac{\partial \kappa}{\partial W_{\text{pca}}} \Big|_{\text{PC}}. \end{aligned} \quad (\text{C.9})$$

Here, the gradient backpropagated through $\mathbf{d}_u$ is accumulated ray-wise in `RenderCUDA`, while the gradient through $\mathbf{o}_u$ is computed Gaussian-wise in `PreprocessCUDA`. These two components are then summed to yield the complete gradient *w.r.t.* the PCA transformation matrix W_{pca} .

C.5 GRADIENTS FOR SCALING AND ROTATION

Thus, for scaling gradients $\frac{\partial \kappa}{\partial \mathbf{s}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}$, we have:

$$\frac{\partial W_{\text{pca}}^{i,j}}{\partial s_k} \Big|_{\text{PC}} = \begin{cases} -\frac{1}{s_k^2} R_{j,k} & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}. \quad (\text{C.10})$$

To derive gradients for rotation $\frac{\partial \kappa}{\partial \mathbf{q}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}$, given the quaternion form $\mathbf{q} = q_r + q_i \mathbf{i} + q_j \mathbf{j} + q_k \mathbf{k}$, we can write the whitening matrix as:

$$W_{\text{pca}} = 2 \begin{pmatrix} \frac{1}{s_1} (\frac{1}{2} - q_j^2 - q_k^2) & \frac{1}{s_1} (q_i q_j + q_r q_k) & \frac{1}{s_1} (q_i q_k - q_r q_j) \\ \frac{1}{s_2} (q_i q_j - q_r q_k) & \frac{1}{s_2} (\frac{1}{2} - q_i^2 - q_k^2) & \frac{1}{s_2} (q_j q_k + q_r q_i) \\ \frac{1}{s_3} (q_i q_k + q_r q_j) & \frac{1}{s_3} (q_j q_k - q_r q_i) & \frac{1}{s_3} (\frac{1}{2} - q_i^2 - q_j^2) \end{pmatrix}. \quad (\text{C.11})$$

As a result, we derive the following end gradients for the quaternion $\mathbf{q}$ as:

$$\begin{aligned} \frac{\partial W_{\text{pca}}}{\partial q_r} \Big|_{\text{PC}} &= 2 \begin{pmatrix} 0 & \frac{q_k}{s_1} & -\frac{q_j}{s_1} \\ -\frac{q_k}{s_2} & 0 & \frac{q_i}{s_2} \\ \frac{q_j}{s_3} & -\frac{q_i}{s_3} & 0 \end{pmatrix}, \quad \frac{\partial W_{\text{pca}}}{\partial q_i} \Big|_{\text{PC}} = 2 \begin{pmatrix} 0 & \frac{q_j}{s_2} & \frac{q_k}{s_2} \\ \frac{q_k}{s_3} & -\frac{q_r}{s_3} & -\frac{2q_i}{s_3} \\ -\frac{2q_j}{s_3} & \frac{q_r}{s_3} & \frac{q_i}{s_3} \end{pmatrix}, \\ \frac{\partial W_{\text{pca}}}{\partial q_j} \Big|_{\text{PC}} &= 2 \begin{pmatrix} -\frac{2q_j}{s_1} & \frac{q_i}{s_1} & -\frac{q_r}{s_1} \\ \frac{q_i}{s_2} & 0 & \frac{q_k}{s_2} \\ \frac{q_r}{s_3} & \frac{q_k}{s_3} & -\frac{2q_i}{s_3} \end{pmatrix}, \quad \frac{\partial W_{\text{pca}}}{\partial q_k} \Big|_{\text{PC}} = 2 \begin{pmatrix} -\frac{2q_k}{s_1} & \frac{q_r}{s_1} & \frac{q_j}{s_1} \\ -\frac{q_r}{s_2} & -\frac{2q_k}{s_2} & \frac{q_j}{s_2} \\ \frac{q_i}{s_3} & \frac{q_j}{s_3} & 0 \end{pmatrix}. \end{aligned} \quad (\text{C.12})$$

C.6 GRADIENTS FOR 3D MEAN

Meanwhile, we compute end gradients *w.r.t.* the Gaussian 3D mean $\boldsymbol{\mu}$ as:

$$\frac{\partial \kappa}{\partial \boldsymbol{\mu}} \Big|_{\text{PC}} = -W_{\text{pca}}^\top \frac{\partial \kappa}{\partial \mathbf{o}_u}. \quad (\text{C.13})$$

C.7 NUMERICAL STABILITY OF OUR DIFFERENTIABLE FRAMEWORK

The choice of forward formulation, backward gradient computation, and intermediate cache variables critically affects numerical stability. Prior approaches such as GOF (Yu et al., 2024b) suffer from backward-propagation instability and must fall back on 3D filtering heuristics to keep training stable. Our closed-form formulation, together with its exact gradient chain, removes this instability at its source and needs no auxiliary filtering whatsoever.



Figure C.1: **Artifacts without 3D filtering in GOF.** (Left) GOF without filtering degenerate Gaussians; (Right) the stable, artifact-free results our forward and backward formulations produce in their place.

The key distinction from our gradient and rendering pipeline lies in that prior methods adopt an alternative expansion of $\mathbf{m}_u^\top \mathbf{m}_u$. Ideally, disregarding numerical issues, the following two forms are geometrically equivalent:

$$\mathbf{m}_u^\top \mathbf{m}_u = \|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2. \quad (\text{C.14})$$

However, when a 3D Gaussian has one axis scaled close to zero, the canonical-space vectors $\mathbf{o}_u$ and $\mathbf{d}_u$ become nearly parallel. In this case, $\cos(\angle(\mathbf{o}_u, \mathbf{d}_u)) \rightarrow 1$, while both $\|\mathbf{o}_u\|$ and $\|\mathbf{d}_u\|$ diverge. This makes the term $\|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2$ highly prone to numerical overflow and, in practice, can even yield negative values and further lead to instable backward propagation as well.

Fig. C.1 (Left) demonstrates the numerical artifacts produced by GOF. When it does not apply additional 3D filtering to remove degenerated Gaussians, severe artifacts appear. The snowflake noise arises from numerical overflow in Eq. C.14 where negative values are prematurely returned and extremely small positive values lead to amplified Gaussian intensity. Since the EWA approximation with Jacobian and conic remains numerically stable, the artifacts mainly stem from inaccurate maximum-response computation, which also induces gridline patterns.

Our CUDA rasterizer, by contrast, computes the cross product and its gradients directly (Eq. B.9 (forward) and Eqs. C.6, C.9 (backward)), which sidesteps this numerical issue entirely. The forward pass is artifact-free and the backward pass yields accurate gradients. Fig. C.1 (Right) makes the point concrete: applied to GOF, our forward and backward formulations eliminate the numerical-instability artifacts even without any 3D filtering to exclude degenerate Gaussians (i.e., flat disk-like or needle Gaussians).

D APPENDIX: ADDITIONAL DETAILS IN ASSOCIATION

Section D.1 provides our detailed derivation. Section D.2 provides implementation considerations required for stable PBF under large-FoV and omnidirectional cameras. Section D.3 clarifies the derivational relationships and explicitly states the conditions under which prior formulas coincide with ours.

D.1 PARTICLE BOUNDING FRUSTUM (PBF) SOLVER

Recall Eq. 8 we define $P = (VH)^\top \in \mathbb{R}^{4 \times 4}$ as the plane transformation matrix from the view space to the canonical space. Take the unknown bounds for $c = \tan \theta_{1,2}$ as example, we have:

$$\mathbf{g}_u = -P_0 + cP_2 = \begin{bmatrix} -P_{00} + cP_{02} \\ -P_{10} + cP_{12} \\ -P_{20} + cP_{22} \\ -P_{30} + cP_{32} \end{bmatrix}, \quad (\text{D.1})$$

where P_i denotes i -th row of the matrix, expressed as a 4×1 column vector, and P_{ij} further denotes j -th element. Recall Eq. 9 assuming we are computing the bounding box of the 1-standard-deviation contour ellipsoid of the Gaussian, i.e., $\lambda = 1$, then we have the following quadratic equation:

$$((\mathbf{f} \circ P_2^\top) \cdot P_2) c^2 - 2((\mathbf{f} \circ P_0^\top) \cdot P_2) c + (\mathbf{f} \circ P_0^\top) \cdot P_0 = 0, \quad (\text{D.2})$$

where $\mathbf{f} = (1, 1, 1, -1)$ is a 1×4 row vector. Here we denote the equation as:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad (\text{D.3})$$

where $\mathcal{T}$ is a 3×3 upper triangular matrix defined with $\mathcal{T}_{ij} = (\mathbf{f} \circ \mathbf{P}_i^\top) \cdot \mathbf{P}_j$.

Similarly, for the bounds of ϕ , we have $c = \tan \phi_{1,2}$ satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{12}c + \mathcal{T}_{11} = 0. \quad (\text{D.4})$$

Interestingly, given a camera-view configuration $[R_c \mid \mathbf{t}_c]$, we find that the parameter matrix $\mathcal{T}$ can be efficiently computed from the Gaussian's view-space covariance $\Sigma_c = R_c R S S^\top R^\top R_c^\top$ and 3D mean $\mu_c = R_c \mu + \mathbf{t}_c$:

$$\mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (\text{D.5})$$

Thus, we can compute the angular bounds (defined by their tangent-space center and extent) from:

$$\begin{aligned} \left(\frac{\tan \theta_1 + \tan \theta_2}{2}, \frac{\tan \phi_1 + \tan \phi_2}{2} \right) &= \left(\frac{\mathcal{T}_{02}}{\mathcal{T}_{22}}, \frac{\mathcal{T}_{12}}{\mathcal{T}_{22}} \right), \\ \|\tan \theta_1 - \tan \theta_2\| &= \frac{2\sqrt{\mathcal{T}_{02}^2 - \mathcal{T}_{22}\mathcal{T}_{00}}}{|\mathcal{T}_{22}|}, \\ \|\tan \phi_1 - \tan \phi_2\| &= \frac{2\sqrt{\mathcal{T}_{12}^2 - \mathcal{T}_{22}\mathcal{T}_{11}}}{|\mathcal{T}_{22}|}. \end{aligned} \quad (\text{D.6})$$

If the discriminant of either quadratic is negative—indicating that the camera's optical center lies inside the Gaussian ellipsoid and the angular bounds are undefined—we conservatively clamp the frustum angles using the camera's maximum field of view (FoV).

Note that this result is directly computed from the 3D view-space true covariance, in contrast to methods like EWA and UT, which rely on intermediate conic approximations in screen space. Our PBF formulation offers both exactness and efficiency.

D.2 DETAILS IN COMPUTING CSF-PBF INTERSECTION

We uniformly sample spherical angles within the maximum field of view (FoV). The resulting x , y , and z coordinates—i.e., the ray direction vectors $\mathbf{d}$ —are projected onto a unit sphere and then transformed into pixel space for inverse color interpolation (see Sec. E). These sampled rays are then grouped according to their corresponding CSFs for subsequent Gaussian association.

In most scenes using a pinhole camera with a limited FoV, the angular bounds in PBF typically lie within $[-\frac{\pi}{2}, \frac{\pi}{2}]$. In such cases, since the $\tan$ function is monotonically increasing over this interval, the intersections between PBF and CSFs can be computed by directly comparing the $\tan$ values of their respective angular bounds. However, in wide FoV scenarios, even when the incidence angle formed by the Gaussian centers remains within $[-\frac{\pi}{2}, \frac{\pi}{2}]$, the angular bounds used in PBF may still fall outside the monotonic region of the $\tan$ function. In these cases, directly comparing $\tan$ values may yield incorrect PBF-CSF intersections, such as mistakenly matching angular bounds as $[\theta'_2, \theta_1]$ (see Fig. D.1 for an example).

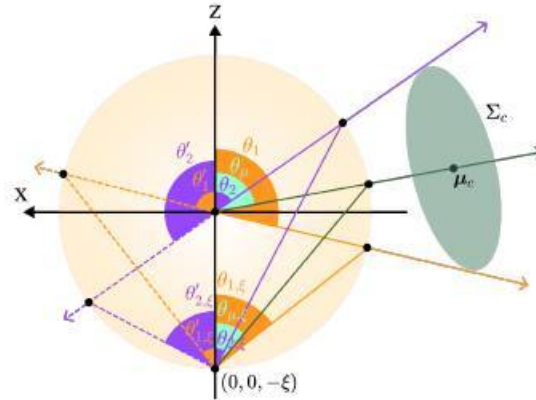


Figure D.1: Mirror Transformation of PBF Angular Bounds. Four candidate mirrored angles $\theta_{1,\xi}, \theta'_{1,\xi}, \theta_{2,\xi}, \theta'_{2,\xi}$ are generated from a given pair of bounds. Given a Gaussian centered at μ_c , the two closest to the corresponding Gaussian's mirrored angle $\theta_{\mu,\xi}$ are used as PBF bounds for robust intersection with mirror-transformed CSF partitions.

To resolve the correct intersections, we introduce a mirror transformation applied to the tangent values of both PBF angular bounds and CSF partitions. Inspired by the CMEI camera model (Mei & Rives 2007), we shift the optical center using a mirror parameter ξ , leading to the following transformation:

$$\tan \theta_\xi = \frac{\tan \theta}{1 + \xi \frac{z}{\|z\|} \sqrt{1 + \tan^2 \theta}} = \frac{\sin \theta}{\cos \theta + \xi}, \quad (\text{D.7})$$

where θ_ξ represents the mirror-transformed angle of θ on the xz -plane, using a mirror parameter ξ . We set $\xi = 1$ throughout all experiments.

Given a pair of tangent value $[\tan \theta_1, \tan \theta_2]$ from our PBF solver (See Equation. D.6), this transformation yields four candidate mirror angles, as illustrated in Fig. D.1. The Gaussian center, in contrast, corresponds to a unique mirror-transformed angle $\theta_{\mu, \xi}$. We exploit this property by selecting the two candidate angles closest to $\theta_{\mu, \xi}$. These two then define the effective lower and upper bounds for PBF, i.e., $[\theta_{1, \xi}, \theta_{2, \xi}]$, which are subsequently compared with CSF partitions to compute the correct intersections.

This transformation guarantees robust, consistent angular comparisons well beyond the monotonic domain of the $\tan$ function. The cost is negligible: our timing analysis (Tab. K.5) shows the full pre-processing pipeline—mirror-angle-based PBF-CSF intersection plus the canonical transformation for all Gaussians—takes under 0.13 ms in total. Our approach thus stays efficient even across thousands of Gaussians in wide-FoV scenarios.

D.3 RELATION TO PRIOR WORKS

Substituting the perspective screen space coordinates:

$$u = \tan \theta, \quad v = \tan \phi$$

into our formulation recovers exactly the expressions in 2DGS (Huang et al. 2024a) and HTGS (Hahlbohm et al. 2025)—precisely as expected, since those methods are the perspective special case of our angular-domain formulation.

Concretely, 2DGS and HTGS start from a screen-space or tangent-plane parameterization and therefore rely on assumptions that hold only for perspective (pinhole) cameras, e.g., geometric identities that arise when rays are implicitly tied to a planar, z -parallel image plane. This assumption holds only for perspective cameras and does not extend to fisheye or omnidirectional systems, where the corresponding surface in screen space is curved. In contrast, our derivation begins from a camera-agnostic angular parameterization in 3D and directly parameterizes Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs). These different starting assumptions change the known quantities and constraints available during derivation, and thus the mathematical conditions under which closed-form bounds and culling guarantees remain valid for generic cameras.

We emphasize that mathematics is a general tool, but deriving these bounds under strictly weaker assumptions—without any reliance on screen-space structure—and validating them with comprehensive experiments across both pinhole and wide-FoV camera families is a core part of our contribution, not an afterthought.

This difference in parameterization also affects the downstream computational pipeline. Gaussian association, tiling/duplication, and sorting depend on how Gaussians map to tiles or bins. Using an angular (CSF-based) domain yields a different bucketing structure compared to pinhole screen-space tiling. This is practically important for large-FoV scenes: under generic cameras a Gaussian’s bound may appear partially “behind” the camera in screen coordinates (see Fig. D.1), causing screen-space methods to produce mirrored or incorrect tangent-based bounds (dashed line in Fig. D.1). To robustly handle these cases, we introduce mirror parameters in the association step (Section D.2) and incorporate the Gaussian center into sorting and correction—components unnecessary in perspective-only derivations.

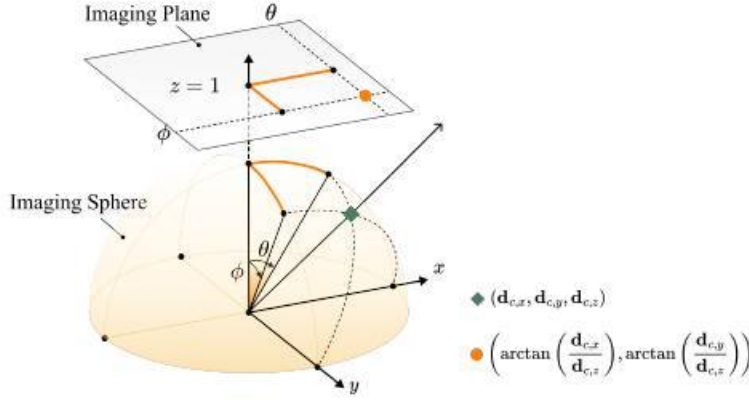


Figure E.1: **Spherical Angles Defining PBF and BEAP.** A view-space direction $\mathbf{d}_c$ is first normalized onto the optical sphere (green spade), followed by angular projections onto the xz and yz planes. The orange point denotes the resulting coordinate (θ, ϕ) on the unfolded BEAP imaging plane.

E APPENDIX: BEAP IMAGE INVERSE INTERPOLATION

To prepare training data, we sample rays to comprehensively cover the original image FoVs and use them as input to our CUDA renderer. Simultaneously, we interpolate the ground truth color for each input ray to ensure accurate and uniform supervision during training.

The projection of sampled rays onto observed images begins with transforming the incidence angle projections θ and ϕ into normalized ray direction vectors $\mathbf{d} = (x, y, z)^\top$, expressed as:

$$x = \frac{\sin \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.1})$$

$$y = \frac{\cos \theta \sin \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.2})$$

$$z = \frac{\cos \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}. \quad (\text{E.3})$$

Several camera models are provided to subsequently project the normalized rays $(x, y, z)^\top$ into pixel space $(x_p, y_p, 1)^\top$.

Pinhole Camera Model. For pinhole camera models, we first have $\frac{x}{z} = \tan \theta$ and $\frac{y}{z} = \tan \phi$. Thus, we have:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.4})$$

KB Fisheye Model. We have the incidence angle φ derived from its projections:

$$r = \sqrt{\tan^2 \theta + \tan^2 \phi} \quad \text{and} \quad \varphi = \arctan(r). \quad (\text{E.5})$$

And we have the fisheye distorted angle:

$$\varphi_d = \varphi(1 + k_1 \varphi^2 + k_2 \varphi^4 + k_3 \varphi^6 + k_4 \varphi^8). \quad (\text{E.6})$$

As a result, the rays are projected onto pixel space, allowing for inverse color interpolation from:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{f_x \varphi_d}{r} & 0 & c_x \\ 0 & \frac{f_y \varphi_d}{r} & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.7})$$

F APPENDIX: HIGH-LEVEL ALGORITHMIC SUMMARY

We provide a high-level overview of our volumetric rendering pipeline (See Alg. 1 and Alg. 2). For clarity, we follow the structure of 3DGS [Kerbl et al. \(2023\)](#) to illustrate differences: newly introduced modules or replaced ones are highlighted in **green**, and significantly modified ones in **yellow**.

ALGORITHM 1: GPU Software Asso. & Render.

```

Function 3DGEER ( $\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V$ ):
  CullGaussian( $\mu, V$ )                                ▷ Frustum Culling
   $F \leftarrow \text{InitCSFs}(w, h, K)$                       ▷ Partition
   $\Sigma_c, \mu_c \leftarrow \text{ViewspaceGaussians}(\mathbf{s}, \mathbf{q}, V)$   ▷ Transform
   $F_{ID} \leftarrow \text{PBF\_Intersect}(\Sigma_c, \mu, F)$         ▷ Sub-frustum Asso.
   $W_{pca} \leftarrow \text{PCA\_Whitening}(\mathbf{s}, \mathbf{q})$               ▷ Transform
   $\mathcal{G}_{ID}, F_K \leftarrow \text{DuplicateWithKeys}(W_{pca}, F_{ID})$ 
                                                         ▷ Indices (Gaussian) & Keys (FrustumID+Depth)
  SortByKeys( $\mathcal{G}_{ID}, F_K$ )                                ▷ Globally Depth Sort
   $R \leftarrow \text{IdentifyFrustumRanges}(F, F_K)$ 
   $I \leftarrow 0$                                           ▷ Init Canvas (w/ Rays)
  for all CSFs  $f \leftarrow I$  do
    for all Rays  $i \leftarrow f$  do
       $r \leftarrow \text{GetFrustumRange}(R, f)$ 
       $\mathbf{m}_u, \mathbf{d}_u \leftarrow \text{Cano\_Ray}(W_{pca}, \mu, i)$       ▷ Transform
       $D_{\mu, \Sigma} \leftarrow \text{Dist}(\mathbf{m}_u, \mathbf{d}_u)$             ▷ Mah-distance
       $I[i] \leftarrow \text{BlendInOrder}(i, \mathcal{G}_{ID}, r, F_K, D_{\mu, \Sigma}, c, \sigma)$ 
                                                         ▷ Alpha Blending
    end
  end
  return  $I$ 

```

ALGORITHM 2: 3DGEER Initialization and Training Framework

w, h : width and height of the training images

K, V : intrinsics and extrinsic meta info.

```

 $\mu \leftarrow \text{SfM Points}$                                 ▷ Init Gaussian Attributes
                                                         ▷ Position
 $\mathbf{s}, \mathbf{q} \leftarrow \text{InitAttributes}()$                   ▷ Scaling, Rotation
 $c, \sigma \leftarrow \text{InitCoefficients}()$               ▷ Color, Opacity
while not converged do
   $\mathbf{r}_c, \hat{C} \leftarrow \text{BEAP}(w, h, K)$                   ▷ View Rays / Colors
                                                         ▷ Asso. & Render
   $C \leftarrow 3DGEER(\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V)$       ▷ See Alg. 1
   $\mathcal{L} \leftarrow \text{Loss}(C, \hat{C})$                           ▷ Loss
   $\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow \text{Adam}(\nabla \mathcal{L})$       ▷ Backprop & Step
  if IsRefinementIteration( $i$ ) then
     $\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow \text{Optimize}(\mu, \mathbf{s}, \mathbf{q}, c, \sigma)$ 
                                                         ▷ Kerbl et al. (2023)
  end
end

```

G APPENDIX: ADDITIONAL IMPLEMENTATION DETAILS

Baselines. We benchmark 3DGEER against the strongest volumetric particle rendering methods that target the projective approximation errors of 3DGS [\(Kerbl et al., 2023\)](#). For large-FoV fisheye inputs, we evaluate against FisheyeGS [\(Liao et al., 2024\)](#), EVER [\(Mai et al., 2025\)](#), and 3DGUT [\(Wu et al., 2025\)](#). Under pinhole settings, we additionally compare with 3DGS and 3DGRT [\(Moënnelocoz et al., 2024\)](#).

Implementation Details. To ensure fair comparison, we align our implementation with the original 3DGS in terms of initialization and hyperparameters on the ScanNet++ and MipNeRF360 datasets. For ZipNeRF, we adopt the training configurations of EVER and SMERF (Duckworth et al. 2024) for all candidate methods. Similar to most ray-based frameworks, our method does not directly access gradients of `mean2D` to control densification or pruning. Instead, we optimize using gradients of the view-space `mean3D`. Unless otherwise noted, all models are trained with consistent supervision strategies and architectural settings. For fisheye datasets (ScanNet++ and ZipNeRF), we precompute bijective grid mappings (Guo et al. 2025) and reproject all rendered outputs to the native KB-Fisheye image space (Kannala & Brandt 2006) before evaluation. Experiments on ScanNet++ and MipNeRF360 are conducted using full-resolution images. For ZipNeRF, due to the high frame count and extremely large fisheye resolution, we use the maximum resolution (see Tab. 3) that allows all baselines to run reliably on a local desktop setup.

Central and peripheral regions in ScanNet++ evaluation. The rays sampled in the central part follows the range:

$$\theta \in [-\frac{w}{2f_x}, \frac{w}{2f_x}], \phi \in [-\frac{h}{2f_y}, \frac{h}{2f_y}].$$

And the Peripheral part takes the rays sampled in remaining angle ranges. Here w, h indicate the width and height of the image and f_x, f_y are the focal lengths defined in the original KB camera model.

Cross-camera generalization evaluation on ZipNeRF. ZipNeRF provides two sets of data: fisheye raw images and their corresponding undistorted frames, which are generated by applying distortion correction and cropping the central regions. Although the frames are aligned on a per-frame basis, camera poses and point clouds were independently reconstructed using COLMAP (Schönbberger & Frahm 2016) for each set. Due to COLMAP’s global optimization independently refining camera trajectories, applying a single rigid transformation to align the two pose sets is insufficient and may introduce alignment errors. To ensure reliable cross-camera evaluation without introducing errors from structure-from-motion (SfM), we always use the same set of test-time camera poses as those used during training. In cross-camera experiments, we vary only the intrinsic parameters, while keeping the extrinsic poses fixed. This ensures fair comparisons without entangling pose inconsistencies arising from preprocessing.

H APPENDIX: EWA SPLATTING PRELIMINARIES

From camera coordinates to ray coordinates. As the coordinate definitions in EWA Splatting (Zwicker et al. 2001), we have the following mapping function from the camera (view) space points $\mathbf{t}$ to the ray space points $\mathbf{x}$ for projective transformation in pinhole camera models, *i.e.*, $\tau: \mathbf{t} \rightarrow \mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \tau(\mathbf{t}) = \begin{bmatrix} t_0/t_2 \\ t_1/t_2 \\ \|(t_0, t_1, t_2)^\top\| \end{bmatrix}, \quad (\text{H.1})$$

$$\mathbf{t} = \begin{bmatrix} t_0 \\ t_1 \\ t_2 \end{bmatrix} = \tau^{-1}(\mathbf{x}) = \begin{bmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{bmatrix}, \quad (\text{H.2})$$

$$, \text{ where } l = \|(x_0, x_1, 1)^\top\|. \quad (\text{H.3})$$

These mappings are not affine, which requires computationally expensive (Westover, 1990; Mao et al. 1995) or locally approximated solutions (Zwicker et al. 2001; Kerbl et al. 2023) to calculate integral in the ray space.

Local affine approximation. EWA splatting by (Zwicker et al. 2001) introduced local affine approximation, denoted as τ_k , to keep the linear effects and the k -indexed reconstruction kernels as Gaussians in 2D screen space. It is defined by the first two terms of the Taylor expansion of τ at the camera space point $\mathbf{t}_k$, then the ray space point $\mathbf{x}$ near around $\mathbf{x}_k$ can be estimated by:

$$\mathbf{x} = \tau(\mathbf{t}) = \mathbf{x}_k + \mathbf{J}_\tau^{\mathbf{t}_k}(\mathbf{t} - \mathbf{t}_k), \quad (\text{H.4})$$



Figure I.1: **Qualitative Results on the Stump (Mip-NeRF360).** 3DGEER outperforms both 3DGS-based (Gaussian Splatting) and 3DPRT-based (Particle Ray Tracing) methods on novel view synthesis under (Left) MipNeRF360 (pinhole) while sustaining high runtime efficiency. The advantage is clearest in the colored boxes.

where $\mathbf{J}_\tau^{\mathbf{t}_k}$ is the Jacobian matrix given by the partial derivatives of τ at the point $\mathbf{t}_k$. For the k -indexed 3D Gaussian $\mathcal{G}_{\Sigma, \mu}$ in a certain camera coordinate, using EWA local affine transformation, the approximated density function can be expressed as a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}$ in the ray coordinate. For $\forall \mathbf{x}$ around $\mathbf{x}_k$, we have:

$$\mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mathbf{x}_k)\right), \quad (\text{H.5})$$

$$\text{where } \Sigma_k = \mathbf{J}_\tau^{\mathbf{t}_k} \Sigma (\mathbf{J}_\tau^{\mathbf{t}_k})^\top. \quad (\text{H.6})$$

Integrating a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}$ along one coordinate axis yields a 2D Gaussian $\mathcal{G}_{\Sigma_k, \hat{\mathbf{x}}_k}^{(2)}$, hence:

$$\int_{\mathbb{R}} \mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}(\mathbf{x}) dx_2 = \mathcal{G}_{\Sigma_k, \hat{\mathbf{x}}_k}^{(2)}(\hat{\mathbf{x}}), \quad (\text{H.7})$$

where $\hat{\mathbf{x}} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$, and the mean $\hat{\mathbf{x}}_k = \begin{bmatrix} x_{k,0} \\ x_{k,1} \end{bmatrix}$, and the 2×2 variance matrix $\hat{\Sigma}_k$ is easily obtained from the 3×3 matrix Σ_k by skipping the third row and column.

Splatting approximation error. Although we can not expect the density function in ray space still obey the Gaussian distribution when we apply general projective transformation to the original view space, exact density can be derived through substitution via Eq. [H.2](#)

$$\begin{aligned} \mathcal{G}_{\Sigma, \mu}(\mathbf{t}) &= \mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) \\ &= \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)^\top \Sigma_k^{-1}(\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)\right), \end{aligned} \quad (\text{H.8})$$

which means that the approximation error can be estimated through Monte Carlo integration of the following equation:

$$\mathcal{E}_k = \int_{\mathbb{R}} |\mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) - \mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x})| dx_2. \quad (\text{H.9})$$

Note that x_2 represents the dimension along the ray direction in the ray coordinate.

I APPENDIX: ADVANTAGE OVER PRIOR RAY-TRACING METHODS

Pure ray-tracing (e.g., 3DGRT) is projective-exact in principle, but in practice it is not: the BVH that 3DGRT uses for intersection tests relies on an inscribed proxy icosahedron for speed, which is tight at the vertices but whose face centers lie $\sim 20\%$ inward by construction. Such proxy shapes degrade intersection accuracy, corrupt the accumulated gradients during density control, and ultimately yield incorrect geometry (e.g., the missing leaves in Fig. [I.1](#)). Worse still, intersection tests for far, tiny, and cluttered Gaussians are bounded by floating-point precision and produce noisy rendering (e.g., Fig. 15 of the 3DGRT paper, whose leftmost native-rendering block shows visible speckles across the tree and background). The gap is telling: even 3DGUT with its UT approximation beats 3DGRT on MipNeRF360. To make the comparison direct, we render the



Figure J.1: **Qualitative Results on the Aria Dataset.** Our method delivers sharper floor textures indoors and clearer distant tree detail outdoors.

Table J.1: **Quantitative comparison on Aria.** Higher PSNR/SSIM and lower LPIPS are better.

Method	Scene		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGEER (Ours)	livingroom	(indoor)	36.398	0.933	0.226
	steakhouse	(outdoor)	30.662	0.881	0.246
FisheyeGS	livingroom	(indoor)	35.686	0.927	0.231
	steakhouse	(outdoor)	29.989	0.869	0.261

same pretrained MipNeRF360-stump through both 3DGRT’s ray-tracing engine and our 3DGEER pipeline. On PSNR $\uparrow$ /SSIM $\uparrow$ /LPIPS $\downarrow$: 3DGRT achieves 25.79/0.745/0.264, ours 26.83/0.774/0.246 (+1.04/+0.029/-0.018)—a uniform win. 3DGEER’s projective-exact, efficient pipeline, optimized with BEAP color supervision, thus delivers both markedly faster speed and higher accuracy (Tab. 4).

J APPENDIX: RESULTS ON EGOCENTRIC DATASET

Recent egocentric devices typically feature wide FoVs, yet existing pipelines fail to hold reconstruction quality at full resolution or when restricted to cropped views. Low-light imaging compounds the difficulty—limited light intake, especially with small sensors and aperture-constrained lenses, drives further degradation.

We evaluate on the Aria egocentric dataset ($\sim 110^\circ$ FoV, circular shape). Our method reaches an average PSNR of 33.5dB, surpassing FisheyeGS (32.8dB) on every metric (Tab. J.1). Qualitatively, our reconstructions retain sharper peripheral detail indoors (Fig. J.1 Left) and recover distant structures such as trees outdoors (Fig. J.1 Right). The +0.7 dB gain sits, as expected, between the improvements on pinhole datasets (MipNeRF360: +0.3dB; ZipNeRF-cross: +2.0dB) and on highly distorted wide-FoV datasets (ScanNet++/ZipNeRF 180° diagonal FoV: +0.9–4.8dB), tracking Aria’s moderate FoV and distortion level.

K APPENDIX: ADDITIONAL MATERIAL AND RESULTS

K.1 ADDITIONAL MATERIAL.

Code, video, and an HTML page are provided in the Supplementary Material.

K.2 ADDITIONAL RESULTS.



Figure K.1: **Out-of-Distribution (OOD) Region in Fig. 6**. 3DGS collapses to black under the numerical singularity $\tan 90^\circ \rightarrow \infty$, whereas our method renders robustly all the way to 180° and reconstructs the green chair that splat-based methods (e.g., FisheyeGS) miss entirely. In the Fig. 6 zoom-in, our method likewise preserves structures such as the corridor and fridge, table and door, and window and cabinet.



Figure K.2: **Extreme-FoV Generalization**. (Left) Our method versus 3DGS under extreme-FoV pinhole (170°) views (trained on undistorted central regions). (Right) Zoom-out of Fig. 6 spanning the full circular 180° FoV, where our method generalizes robustly to out-of-distribution regions.

Table K.1: **MipNeRF360 Quantitative Results by Sequence**. The overall results are shown in Tab. 4

Method	Metric	Bicycle	Bonsai	Counter	Garden	Kitchen	Stump	Flowers	Room	Treehill
Ours	PSNR $\uparrow$	25.63	32.23	29.05	27.60	31.78	26.83	21.83	32.09	22.83
	SSIM $\uparrow$	0.774	0.946	0.910	0.860	0.931	0.774	0.613	0.946	0.637
	LPIPS $\downarrow$	0.223	0.175	0.195	0.136	0.120	0.246	0.340	0.108	0.344

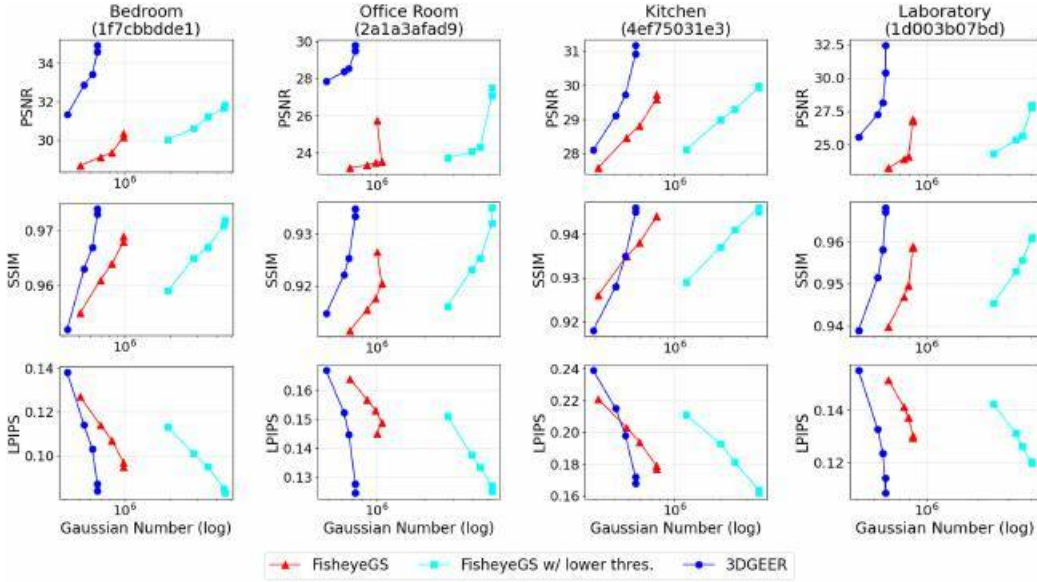


Figure K.3: **PSNR $\uparrow$ SSIM $\uparrow$ LPIPS $\downarrow$ Trends When Scaling-Up Gaussian Counts.** Across all three metrics, brute-force scaling never closes the projective-exactness gap.

Table K.2: **ScanNet++ Quantitative Results by Sequence (Full FoV).** The overall results are shown in Tab. 2.

Scene	EVER			FisheyeGS			3DGUT			3DGEER (Ours)		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.24	0.916	0.164	24.74	0.932	0.146	29.26	0.934	0.147	29.18	0.940	0.141
Office Room	27.80	0.899	0.169	25.75	0.927	0.145	29.37	0.928	0.145	29.78	0.935	0.125
Bedroom	32.60	0.948	0.131	31.95	0.970	0.095	34.65	0.967	0.112	34.94	0.974	0.084
Kitchen	29.77	0.924	0.213	29.72	0.944	0.177	30.78	0.941	0.196	31.17	0.946	0.168
Laboratory	28.96	0.936	0.156	26.91	0.959	0.129	29.12	0.948	0.151	32.43	0.968	0.114

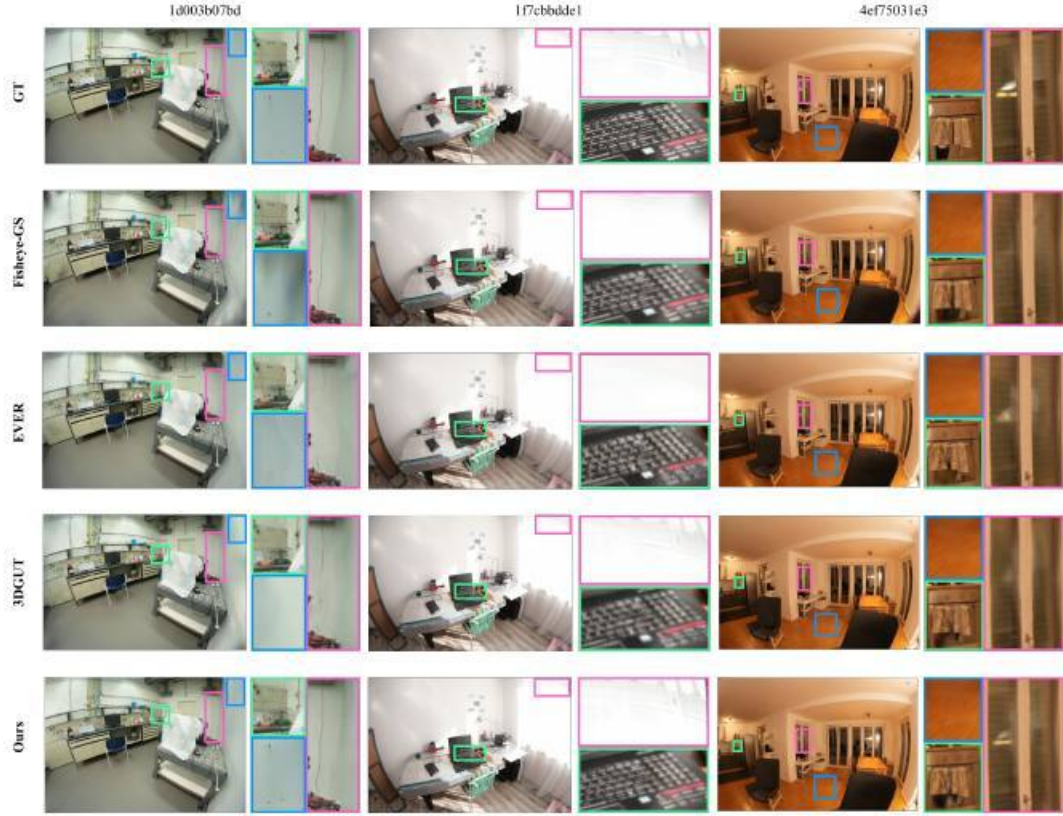


Figure K.4: **Qualitative Results on the ScanNet++ Dataset.** The gap is clearest in the zoomed-in colored boxes, especially in structural detail and artifact regions. Our method reconstructs the periphery far better than FisheyeGS, shows fewer artifacts and sharper detail than EVER, and produces crisper object boundaries and textures—wires, keyboards, wooden floors—than 3DGUT.

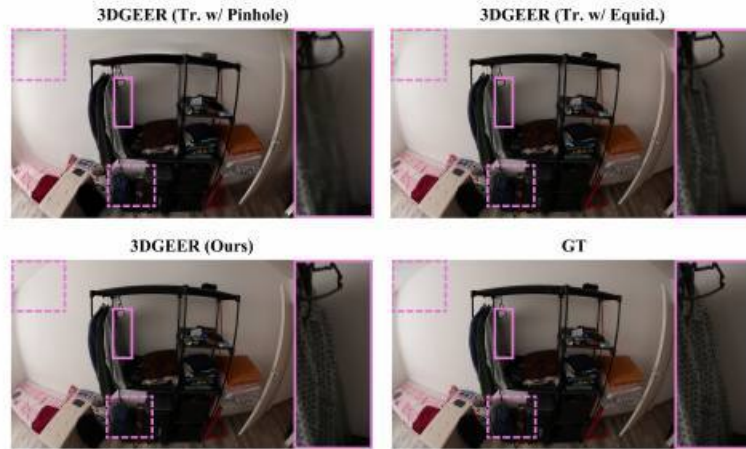


Figure K.5: **Impact of BEAP supervision.** On ScanNet++, BEAP’s balanced angular sampling yields smoother gradients and preserves geometric and texture detail markedly better than pinhole or equidistant sampling (full quantitative stats in Tab. [K.8](#)).

Table K.3: **Quantitative Results on the ScanNet++**. PSNR $\uparrow$ SSIM $\uparrow$ LPIPS $\downarrow$ Trends When Scaling-Up Gaussian Counts. Bedroom, Office Room, Kitchen, and Laboratory in order.

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	28.67	0.955	0.127	512K	30.03	0.959	0.113	1914K	31.33	0.952	0.138	422K
7k	29.12	0.961	0.114	695K	30.61	0.965	0.101	2829K	32.87	0.963	0.114	543K
10k	29.36	0.964	0.107	827K	31.21	0.967	0.095	3489K	33.42	0.967	0.103	616K
20k	30.17	0.968	0.097	985K	31.68	0.971	0.085	4467K	34.59	0.973	0.087	664K
30k	30.36	0.969	0.095	985K	31.84	0.972	0.083	4535K	34.94	0.974	0.084	664K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.19	0.911	0.164	677K	23.72	0.916	0.151	2778K	27.84	0.915	0.167	480K
7k	23.33	0.916	0.157	867K	24.06	0.923	0.138	3936K	28.37	0.922	0.152	621K
10k	23.45	0.918	0.153	983K	24.31	0.925	0.133	4481K	28.54	0.925	0.145	668K
20k	23.51	0.921	0.149	1072K	27.07	0.932	0.127	5272K	29.49	0.933	0.128	732K
30k	25.75	0.927	0.145	1001K	27.50	0.935	0.125	5272K	29.78	0.935	0.125	732K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	27.58	0.926	0.221	314K	28.12	0.929	0.211	1191K	28.11	0.918	0.239	292K
7k	28.47	0.935	0.203	485K	28.99	0.937	0.193	1997K	29.11	0.928	0.215	410K
10k	28.82	0.938	0.194	588K	29.29	0.941	0.181	2491K	29.73	0.935	0.198	476K
20k	29.60	0.944	0.179	760K	29.93	0.946	0.164	3563K	30.91	0.945	0.172	552K
30k	29.72	0.944	0.177	760K	29.97	0.945	0.162	3563K	31.17	0.946	0.168	552K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.27	0.940	0.152	602K	24.32	0.945	0.142	2429K	25.59	0.939	0.155	406K
7k	23.94	0.947	0.141	740K	25.36	0.953	0.131	3258K	27.27	0.952	0.133	524K
10k	24.12	0.950	0.137	788K	25.65	0.956	0.126	3569K	28.14	0.958	0.124	562K
20k	26.76	0.959	0.130	838K	27.77	0.961	0.120	4035K	30.39	0.967	0.108	582K
30k	26.91	0.959	0.129	838K	27.97	0.961	0.120	4035K	32.43	0.968	0.114	582K

Table K.4: **Runtime Analysis vs. Other Methods**. 3DGEER, compared against efficient splatting-based methods on MipNeRF360 and ScanNet++, matches or beats them in total runtime despite its exact ray-based rendering, as the stage-wise breakdown makes clear.

Dataset	Method	Avg. Timings (ms) $\downarrow$				
		Prep.	Dup.	Sort	Render	Total
MipNeRF360	3DGS	0.32	0.27	0.65	1.40	2.92
	3DGEER (Ours)	0.37	0.17	0.27	2.10	3.06
ScanNet++	FisheyeGS	0.10	0.63	1.45	2.33	4.70
	3DGEER (Ours)	0.13	0.26	0.59	2.89	3.98

Table K.5: **Runtime Analysis: Asso. Method on ScanNet++**. PBF-based association is pitted against splatting-based EWA and UT—even with the SnugBox accelerator—inside the 3DGEER pipeline. PBF is fastest across every stage.

3DGEER Asso.	Prep.	Avg. Timings (ms) ↓				Total	Avg. FPS ↑
		Dup.	Sort	Render			
EWA	0.13	1.17	3.27	12.69	17.84	56	
+ SnugBox	0.13	0.60	1.42	6.22	8.68	115	
UT	0.13	0.71	1.94	7.97	11.16	90	
+ SnugBox	0.13	0.28	0.64	3.16	4.38	228	
PBF (Ours)	0.13	0.26	0.59	2.89	3.98	251	

Table K.6: **Impact of PBF-CSF Association**. We evaluate with the artifact-sensitive LPIPS metric and probe cross-model robustness by testing each variant on models trained under the other association strategies. We also report rendering memory cost, the average number of Gaussian associations per tile, and the in-method vs. cross-method gap Δ LPIPS. PBF wins on every axis, and its near-zero Δ LPIPS confirms it is the only association that stays consistent across models.

Asso. Method	Gaus. ↓ per Tile		Mem. ↓ (GB)	LPIPS ↓	Cross-LPIPS ↓	Δ LPIPS ↓ (1e-2)
	mean	std				
EWA	2203	1447	2.2	0.1250	0.1321	0.71
UT	1377	1058	1.4	0.1249	0.1302	0.53
PBF (Ours)	475	427	0.63	0.1245	0.1251	0.06

Table K.7: **Training-Time Transmittance Func. Replacement**. The overall results are shown in Tab. 5.

Scene	3DGS Splats + PBF				FisheyeGS Splats + PBF				3DGEER (Ours)			
	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)
Storage	19.27	0.859	0.173	1293	24.75	0.933	0.150	781	29.18	0.940	0.141	429
Office Room	20.89	0.848	0.182	1527	25.89	0.928	0.150	1119	29.78	0.935	0.125	732
Bedroom	27.39	0.923	0.119	1542	31.96	0.970	0.096	1079	34.94	0.974	0.084	664
Kitchen	25.15	0.890	0.239	1241	29.87	0.949	0.182	801	31.17	0.946	0.168	552
Laboratory	21.60	0.869	0.173	1379	27.03	0.962	0.131	823	32.43	0.968	0.114	582

Table K.8: **Impact of Different Supervision Space**. The overall results are shown in Tab. 6.

Supervision Space Scene	3DGEER w/o BEAP Central Perspective			3DGEER w/o BEAP Perspective			3DGEER w/o BEAP Equidistant			3DGEER (Ours) BEAP		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.42	0.931	0.143	19.74	0.834	0.302	29.03	0.938	0.142	29.18	0.940	0.141
Office Room	28.90	0.927	0.133	21.12	0.825	0.291	29.71	0.932	0.126	29.78	0.935	0.125
Bedroom	33.86	0.965	0.090	24.38	0.882	0.240	34.60	0.973	0.096	34.94	0.974	0.084
Kitchen	30.65	0.941	0.172	23.23	0.889	0.338	31.23	0.941	0.177	31.17	0.946	0.168
Laboratory	27.35	0.951	0.118	17.08	0.834	0.329	30.65	0.956	0.135	32.43	0.968	0.114

Table K.9: **ZipNeRF Quantitative Results by Sequence.** Alameda, Berlin, London, and NYC in order. The overall results are shown in Tab. 3

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	20.74	23.64	0.820	0.802	0.223	0.277	17.39	23.74	0.736	0.839	0.265	0.240
EVER	22.46	23.06	0.852	0.777	0.144	0.274	20.06	22.83	0.789	0.794	0.213	0.250
3DGUT	21.78	22.39	0.841	0.718	0.193	0.368	17.03	22.58	0.588	0.765	0.329	0.311
Ours	24.60	25.00	0.857	0.839	0.154	0.241	21.49	25.21	0.792	0.807	0.232	0.277

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.25	28.25	0.895	0.924	0.206	0.209	20.08	27.86	0.848	0.929	0.233	0.194
EVER	25.26	26.97	0.912	0.907	0.155	0.221	23.18	27.72	0.877	0.913	0.196	0.212
3DGUT	25.37	27.02	0.904	0.876	0.197	0.288	18.57	27.43	0.730	0.898	0.297	0.249
Ours	25.51	29.15	0.916	0.925	0.166	0.194	23.58	29.51	0.878	0.915	0.219	0.228

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.57	26.65	0.829	0.861	0.253	0.254	20.83	27.33	0.768	0.883	0.288	0.229
EVER	24.59	26.83	0.823	0.859	0.218	0.223	23.14	26.98	0.793	0.860	0.268	0.235
3DGUT	25.61	26.25	0.854	0.803	0.221	0.336	19.82	26.41	0.669	0.833	0.313	0.291
Ours	26.60	27.84	0.895	0.882	0.142	0.230	23.95	27.15	0.840	0.846	0.217	0.289

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.15	27.21	0.890	0.885	0.162	0.215	19.41	27.50	0.811	0.907	0.203	0.183
EVER	28.39	27.71	0.931	0.860	0.095	0.228	25.22	28.27	0.879	0.883	0.154	0.192
3DGUT	26.31	26.69	0.917	0.817	0.121	0.302	19.00	27.42	0.659	0.866	0.263	0.228
Ours	28.26	28.49	0.922	0.906	0.097	0.190	24.52	28.55	0.874	0.885	0.168	0.220